Technische Universität
Kaiserslautern

Department of Computer Science

Master's Thesis

Scaling out Equi Joins over Massive Semi-Structured Data Streams

Author:
Damjan Gjurovski

March 2019

Supervisor:
Prof. Dr.-Ing. Sebastian Michel

Reviewers:
Prof. Dr.-Ing. Sebastian Michel
MSc. Manuel Dossinger

Abstract

In this work, we address the problem of joining huge volumes of JSON documents, that do not have to be about the same topic and as such consist of various types of parameters. Nowadays, working with this big amount of data is becoming a reality for more and more different businesses. Handling this data requires a distributed approach, that will be able to parallelize the work and scale efficiently over time whilst ensuring that the work is efficiently distributed among the machines, such that the same documents will be replicated as little as possible.

As a result, we propose a novel approach, focused on performing real-time equi joins over the JSON documents provided as input, implemented in Apache Storm. Since the content of the documents is unknown, we cannot specify the join parameters in advance. Instead, by analyzing the attributes of the documents, our algorithm determines whether they have shared elements and thus, whether they can be joined or not. Mainly, our approach consists of two parts. The goal of the first part is to spread the data among the available machines such that, the same data is sent as few times as possible and every machine works on approximately the same number of documents. To satisfy these needs, an efficient and innovative partitioning algorithm based on discovering association groups is implemented. Once the partitions are created and allocated to the available machines, the second part of the approach, responsible for finding the joinable documents is initiated. The join algorithm, as a foundation uses a Frequent Pattern Tree (FPTree), which represents another idea from the context of association rule mining.

Through experiments, we compare our partitioning algorithm with already existing partitioning approaches and we show that our approach provides better load distribution and replication factor than the competitors and as such is more suitable when working with vast JSON documents. Additionally, by direct comparison, we show that our proposed join algorithm achieves superior time than the normally used join algorithms when working with JSON documents in which there is a common attribute present in most of them. The mentioned experiments are realized using a recognized benchmark, called NoBench.

Zusammenfassung

In dieser Arbeit betrachten wir das Problem der verteilten Joinberechnung über Datenströmen bestehend JSON-Dokumenten. Solche semi-strukturierten Daten sind heutzutage in vielen Bereichen anzutreffen und besitzen, im Vergleich zu traditionellen relationalen Daten, kein festes Schema. Ziel der Joinberechnung ist es verwandte Dokumente zusammenzuführen, um weiterführende interessante Erkenntnisse zu erlangen. So können beispielsweise Twitter Tweets und Logs von Änderungen in Wikipedia zusammengeführt werden, um zu erkennen welche aktuellen Events Einfluss auf bekannte Entitäten haben. Damit die Verarbeitung auch mit großen Datenmengen, Stichwort Big-Data, umgehen kann, soll sie verteilt auf einer größeren Anzahl von Knoten eines Rechenclusters laufen. Dabei müssen die einzelnen JSON-Dokumente so auf die Knoten verteilt werden, dass möglichst wenig Redundanz anfällt und dennoch das korrekte Joinergebnis berechnet werden kann.

Wir präsentieren einen neuartigen Ansatz, fokussiert auf die Berechnung von Equi-Joins in Echtzeit, implementiert basierend auf dem Apache Storm Framework. Da die Dokumente keinem festen Schema unterliegen, ist es nicht möglich feste Joinattribute festzulegen, worüber traditionell die Partitionierung erfolgen würde. Unser Ansatz analysiert daher die zugrunde liegenden Dokumente und findet, dynamisch, übereinstimmende und sich widersprechende Belegungen von Attribut-Wert-Paaren, auf Grund derer ein Joinergebnis generiert werden kann. Der vorgeschlagene Ansatz besteht aus zwei Teilen: Zuerst müssen Dokumente basierend auf ihren Inhalten auf die bereitstehenden Maschinen verteilt werden, wobei darauf geachtet wird, dass ein Dokument möglichst gering repliziert wird und, dass jede Maschine ungefähr die gleiche Anzahl an Dokumenten erhält. Um dies zu erlauben, schlagen wir einen innovativen Partitionierungsalgorithmus basierend auf Assoziationsgruppen vor. Sobald die berechneten Partitionen den bereitstehenden Maschinen zugeteilt wurden, startet Phase zwei unseres Ansatzes, der Berechnung der lokalen Joinergebnisse, auf jeder Maschine. Der dort verwendete Algorithmus basiert auf dem Konzept der Frequent-Pattern-Trees (FP-Trees), welches, ebenso wie die initiale Partitionierung, aus dem Bereich der Assoziationsanalyse aus dem Gebiet des Data-Minings stammt.

Anhand von Experimenten evaluieren wir die Tauglichkeit der Partitionierung im Vergleich zu bestehenden Ansätzen und sehen, dass der vorgeschlagene Ansatz sowohl bessere Lastverteilung als auch geringere Datenreplikation aufweist und somit für die Verarbeitung von sehr großen Datenmengen geeignet ist. Zusätzliche Experimente zeigen darüber hinaus, dass der präsentiere lokale Joinalgorithmus Leistungsvorteile gegenüber alternative Ansätzen besitzt, wenn die meisten Dokumente ein gemeinsames Attribut besitzen. Die Experimente wurden auf dem publizierten NoBench Standarddatensatz ausgeführt.

Contents

1 Introduction **1**
1.1 Problem Statement & Sketch of the Approach 2
1.2 Contribution 3

2 Background **5**
2.1 Data Stream Processing 5
2.2 Apache Storm 5
2.2.1 Topologies 6
2.2.2 Storm Cluster 6
2.2.3 Parallelism in Storm Topology 7
2.2.4 Stream Groupings 8
2.2.5 Windows 8
2.3 Association Analysis 9
2.4 Frequent Pattern Trees 10
2.5 NoBench Data Generator 13

3 Related Work **15**
3.1 Distributed Stream Joins 15
3.1.1 BiStream 15
3.1.2 Approximate Distributed Stream Joins 16
3.1.3 Photon 16
3.2 Partitioning 16
3.2.1 Tracking Set Correlations at Large Scale 17

4 Approach **19**
4.1 Naive Approaches 19
4.2 Topology for Joining JSON Documents 21
4.3 Updating Partitions 25

5 Partitioning Approaches **27**
5.1 Disjoint Sets 28
5.2 Set-Cover–Based Algorithms 29
5.3 Association Groups Based Algorithm 33

6 Join Algorithms for Local Execution **39**
6.1 Nested Loop Join (NLJ) 39
6.2 Hash-Based Join (HBJ) 40
6.3 FPTree-Based Join 42
6.3.1 FPTree Design and Creation 42
6.3.2 FPTreeJoin Algorithm for Finding Join Partners 46
6.3.3 Distributed Implementation 49

7 Experiments **51**
7.1 Setup 51
7.2 NoBench Data 51
7.3 Configuration Parameters 52
7.4 Local Experiments 53
7.4.1 Processing Load 53

7.4.2 Replication Factor 54
7.4.3 Execution Time for Partitioning Approaches 56
7.4.4 Execution Time for Local Join Approaches 57
7.5 Experiments in Distributed Setting 58
7.5.1 Processing Load 58
7.5.2 Replication Factor 60
7.5.3 Handling High Volume of Documents 62
7.5.4 Impact of Sliding Window 63

8 Future Work **65**
8.1 Recalculating Partitions 65
8.2 Similarity Joins 66

9 Conclusion **69**

List of Figures

1.1 Example for Joinable Documents 2

2.1 Word Count Topology 6
2.2 Parallelism Example 8
2.3 Sliding Window Example, from [2] 9
2.4 Tumbling Window Example, from [2] 9
2.5 Pruning in Apriori Algorithm 11
2.6 Frequent Pattern Tree Example 12

3.1 Topology for Finding Tag Correlations 17

4.1 Naive Approach 1 19
4.2 Naive Approach 2 20
4.3 Join Approach Topology 21

5.1 Finding Association Groups 34

6.1 Constructing an FPTree 45

7.1 Processing Load - Local Measurements 54
7.2 Replication Factor - Local Measurements 55
7.3 Partitioning Approaches Execution Time – Local Measurements 56
7.4 Join Approaches Execution Time - Local Measurements 57
7.5 Processing Load 59
7.6 Maximal Number of Documents per Machine 60
7.7 Replication Factor 61
7.8 Changing the Number of Documents 62
7.9 Changing the Number of Documents - Execution Time 63
7.10 Unseen Documents for Varying Sliding Window 64

List of Algorithms

1 Disjoint Sets Algorithm (**DS**) 28
2 Set Cover Based Algorithms - Creation of Initial Partitions . . . 30
3 Set Cover Based Algorithms - Phase 2: Assigning the Remaining Tagsets by Optimizing the Communication Overhead (**SCC**) . . 31
4 Set Cover Based Algorithms - Phase 2: Assigning the Remaining Tagsets by Optimizing the Processing Load (**SCL**) 32
5 Assigning Unseen Documents to Partitions 33
6 Association Groups Based Algorithm - Phase 1: Finding Association Groups(**AG-P1**) 35

7 Association Groups Based Algorithm - Phase 2: Finding Partitions(**AG-P2**) 36
8 Association Groups Based Algorithm - Consolidation of Association Groups in Merger Bolt (**AG-MC**) 37
9 Nested Loop Join (**NLJ**) 39
10 Hash-Based Join (**HBJ**) 41
11 Ordering of Key-Value Pairs 43
12 FPTreeJoin 46
13 FPTreeJoin - Part 1: Checking Candidate Branches 47
14 FPTreeJoin - Part 2: Delivering Join Partners 48

List of Tables

2.1 Example Market Basket Data 10

6.1 Set of JSON Documents 43

8.1 Example Set of JSON Documents 67

Chapter 1

Introduction

Working with virtually never-ending flows of data, called data streams, is a research field that has been extensively studied and analyzed by a variety of scholars and researchers. The increased interests in social media and interconnection of computing devices over the internet that unceasingly create data which cannot be gathered, processed, and evaluated using the traditional approaches, pave the way for *Data Stream Processing*. Moreover, many businesses for marketing and advertisement require real-time analysis of vast amounts of information, which further strengthens the need for analysis techniques that can work on the data as it arrives. Through real-time analysis, valuable insights from the data can be gathered, which, if not retrieved immediately, may be forever lost. Additionally, as opposed to batch processing, there is no need to worry that complementary information may be split and processed in different batches. All of the relevant information for a particular time point will always be handled together.

Performing analysis on enormous, continuous data requires a distributed approach. Working in a distributed environment imposes new challenges such as to make sure that even though every machine will handle a portion of the data, correct results will be provided, that there is acceptable replication of the data among the machines and that the machines are balanced in terms of load. For that reason, efficient partitioning or clustering algorithms that can scale for vast amounts of data are required. Furthermore, performing joins in a distributed environment is of great interest. Through joins, information from several different sources can be grouped together, providing more meaningful data that can be of great benefit during analysis. One example could be analyzing user clicks for determining the effectiveness of an advertisement in a particular country. Through the analysis of the user clicks a company can determine if the advertisement was successful and with it attracted new clients. This can be realized by joining the users that clicked the advertisement with the existing users that have already made a purchase from the company. This advertisement can be particularly useful for the company to determine how to redistribute the budget for further promotions. Moreover, by having information about the loyal customers, through the real-time join of the user clicks with the existing users, promotions to the loyal customers can be provided. Although performing joins in a distributed environment is a deeply investigated topic, to the best of our knowledge, most of the research is focused on performing traditional SQL-like joins where the join parameters are either known in advance or provided at query time.

In our work, the main focus is on performing joins in a distributed environment on top of JSON [3] documents. The equality join definition differs from other research because we do not know the join partners in advance instead, the JSON documents are analyzed and we determine if they share at least one key-value pair. Additionally, in our definition of the join operation, we require that the

documents do not have more conflicting values than a specified threshold, for the same keys. This means that two documents can be joined if they share at least one common key-value pair and for every equal key they do not have more conflicting values than the specified threshold. While searching for documents that can be joined, we ignore the different number of key-value pairs or unique keys that the documents may have. If the join condition is satisfied for at least one key, then we produce those documents as resulting joinable documents. For performing efficient and correct joins we heavily rely on the partitioning strategy. Through appropriate partitioning, all of the potentially joinable documents will be located on the same machine and the algorithm for performing the join will be executed in a more timely manner. By producing as output all of the possibly joinable documents, we create the possibility for analyzing documents that contain complementary information from different sources, without knowing the join parameters in advance.

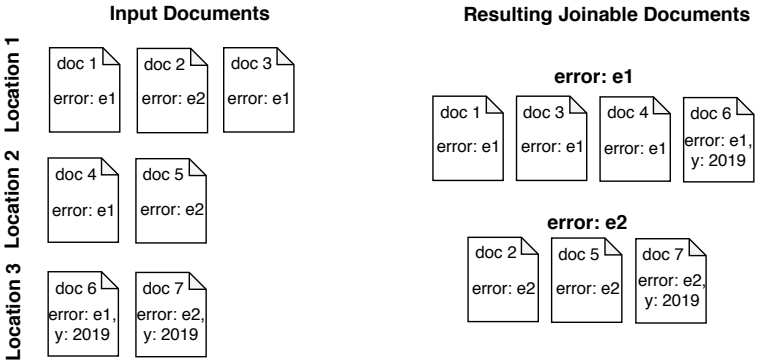


Figure 1.1: Example for Joinable Documents

One example set of documents is given in Figure 1.1. The documents represented in the figure are example server logs. If we would like to analyze the same errors that have occurred in different locations or perform other analysis, we would have needed to inform the application for the different queries in advance. Through our approach all the possibly joinable documents will be grouped and together with the join parameter(s) provided as output in real-time, eliminating the need for beforehand knowledge of the join parameters. This facilitates the job of the analyst since there is no need to know all of the possible types of errors that can occur.

1.1 Problem Statement & Sketch of the Approach

We consider a stream of JSON documents D where every document $d_i \in D$ consists of set of key-value pairs $kv_{d_i} = \{k_1 : v_1, k_2 : v_2, ...\}$. Real-time analysis over a stream of documents requires processing of the documents in parallel. For that reason, the stream of JSON documents is distributed over a cluster of machines. The distribution of the given set of documents is realized by performing partitioning based on the key-value pairs. As a result, k partitions

$pr_1, .., pr_i$ are created. The partitioning is performed over a count-based sliding window. The partitioning needs to ensure that:

- The replication factor of the documents across the machines is tolerable, thereby keeping the network traffic low
- The documents are equally distributed among the machines, leading to acceptable load balancing of the nodes

Additionally, we need to guarantee that for every document in the given count-based window, we will deliver all of the join partners. For that reason, the partitioning should be realized in a way that will make sure, that if a document is found to be present in more than one partition, it will be propagated to all of the machines responsible for these partitions. By computing the join partners locally at every machine, we make sure that no join partner will be forgotten.

Once the partitions are created and distributed to the machines, every machine is responsible for finding the joinable documents from the allocated partition (pr_i). For finding the joinable documents we use a threshold θ. We say that two documents are joinable if they have at least one common key-value pair and for all equal keys, they do not have more conflicting values than the specified threshold θ. Formally, the join can be defined as:

$$(\exists kv_i : kv_i \in d_1, d_2) \wedge (\forall k_1, v_1, k_2, v_2, k_1 v_1 \in d_1 \wedge k_2 v_2 \in d_2 \wedge k_1 = k_2 : |v_1 \neq v_2| < \theta) \tag{1.1}$$

The results from the join computation are stored locally on every machine. For providing all the join partners for the particular count-based window, the results from all machines are grouped.

1.2 Contribution

The main result of this work is a novel framework for distributed join processing. For realizing the proposed architecture, we first examine several different, existing partitioning algorithms. We develop our own partitioning algorithm based on *Association Groups*, compare the algorithm developed in this thesis with the existing approaches, and present the results. Furthermore, we investigate several different types of joins. We construct a novel join based on *Frequent Pattern Trees* and we measure it against the other types of join. We analyze the findings and explain in detail why the other join approaches are not suitable for our scenario. The comparison of the partitioning and join approaches is performed in both local and distributed setting. In the end, both the partitioning and join algorithm are implemented in a distributed environment using Apache Storm and precise measurements of the replication, load balance and execution time are presented.

Chapter 2

Background

2.1 Data Stream Processing

In today's information era, excessive amounts of data drives and influences the businesses. Analyzing and utilizing the huge amount of data that is created continuously, with high velocity and variety is the main challenge. Using traditional tools and techniques is no longer suitable since they cannot provide acceptable approaches for generating insights in a timely manner. Additionally, in many scenarios the data is enormous and it is not possible to store it. Moreover, querying the data streams is different from querying the traditional database management systems (DBMS). There is a need for continuous queries which are not supported by the traditional DBMS. The focus lies in using innovative and novel approaches for analyzing and processing of Big Data.

Data Stream Processing is a technology, that enables querying continuous data streams in order to gain insights and answer questions within a small time period after the data is received [11]. Stream processing works with data streams which represent potentially never-ending flows of data. Storing a potentially unbound data stream is impossible, for that reason the stream processing is carried out on data elements directly, as they arrive. In order to control the input data, stream processing is carried out on *windows* over the data which are further described in Section 2.2.5. One of the main characteristics of stream processing is that there is a single scan of the data. This means that once a record is seen and processed it will no longer be available for analysis. This makes stream processing the appropriate approach when approximate answers are satisfactory.

Compared to batch processing, stream processing processes the data as it arrives, online, whereas batch processing gathers the data over time and it processes big chunks of data. This means that stream processing requires much less hardware. In many situations storing large batches of data and analyzing them in a later point in time is not enough and real-time analysis of the data elements is required. For example, detecting frauds in banking or other similar systems. This information needs to be made available while the fraud is happening so that it can be possible to stop the fraud before it happens.

2.2 Apache Storm

One of the most prominent frameworks for real-time processing of unbounded streams that carry large volumes of data is Apache Storm [1]. This distributed computation system, enables scalability and can run parallel computations across a cluster of machines. Storm is easy to operate and also it can be implemented in various programming languages. Additionally, Storm is fault-tolerant, the

nodes of the system are automatically restarted after they fail and also it is reliable, by guaranteeing that every message will be processed at least once.

The applications in Apache Storm are grouped into so-called Storm topologies. Every storm topology consists of Spouts and Bolts that work together in order to execute the logic of the application. The connection between the Spouts and Bolts is realized by using stream groupings. Apache Storm is a widely used framework for real-time analysis that is used by many technological giants such as Yahoo, Twitter, Spotify, and others.

2.2.1 Topologies

A Storm topology defines how the data stream is going to be processed by connecting Spouts and Bolts. The data stream is read and provided as input to the topology through the Spout. The Spout can read the data stream from a database, from an external source or from an API that provides continuous data. The Bolts are the elements of Storm that do the processing of the data stream. They are connected to a Spout, from which they receive tuples. After receiving the tuple, the Bolts perform operations on top of them and they provide an output. This output can go to a different Bolt for further processing, which means that it is possible to connect Bolts with other Bolts. This gives the possibility of creating complex topologies that can tackle various processing problems.

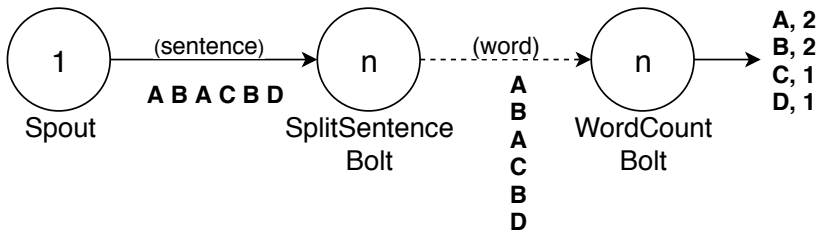


Figure 2.1: Word Count Topology

An example for a Storm topology is presented in Figure 2.1. The topology has one spout, responsible for generating the sentences which will be provided as input to the topology, and two bolts. The *SplitSenctence* bolt is subscribed to the Spout and his job is to split the sentences that he receives as input, into words. The *WordCount* bolt is subscribed to the SplitSentence bolt using fields grouping (explained in Section 2.2.4) on the key *word*. This means that all the same words that appear in different sentences will always be sent to the same instance of the WordCount bolt. The job of this bolt is to count how many times every word has appeared in the sentences.

2.2.2 Storm Cluster

Storm Cluster represents the cluster of distributed machines that work together to execute the storm topologies. The Storm Cluster consists of two different

types of nodes: master node and worker nodes. The master node runs a daemon program called Nimbus. The Nimbus is responsible for distributing the code around the cluster of machines, assigning tasks to the machines and scanning for failures. The daemon called Supervisor runs on every worker node. The Supervisor starts and stops worker processes depending on the assignments of the Nimbus. The worker processes execute parts of the topology. The coordination between the Nimbus and the Supervisor is carried out by the Zookeeper. The Zookeeper is responsible for keeping the states of the Nimbus and the Supervisor. This means that if either of them fails, they can be restarted without affecting the topology.

2.2.3 Parallelism in Storm Topology

Apache Storm consists of three main things that are used to run a topology: worker processes, executors and tasks. In the Storm Cluster, the worker processes execute parts, subsets of the topology. One topology consists of multiple processes that run on different machines in the Storm Cluster. Every worker process runs one or several executors for one or more Spouts or Bolts of the topology. An executor represents a thread that is made available for the worker process. Every executor runs an arbitrary number of tasks for the same Bolt or Spout. A task performs the actual Spout or Bolts code implemented by the developer. It is important to note that the number of tasks is the same throughout the existence of the topology, but the number of executors may change.

Apache Storm provides freedom by enabling developers to specify the parallelism for their topology. Developers can configure the number of worker processes, executors and tasks that they want for their topology. The configuring can be performed using the following methods:

- *setNumWorkers()* - specifies the number of worker processes for the whole topology
- *setBolt(..,numExecutors) / setSpout(..,numExecutors)* - the number of executors can be given as parameter directly when the Bolt or Spout is added as a component to the topology
- *setNumTasks()* - specifies the number of tasks for the component (Spout or Bolt)

Figure 2.2 represents an example topology setting. Two components are implemented: BlueSpout and GreenBolt. We want the topology to consists of two worker processes.

```
configuration.setNumWorkers(2)
```

For the BlueSpout, we want to have two executors and one task per executor.

```
topology.setSpout("blue", new BlutSpout(), 2).setNumTasks(1)
```

For the GreenBolt, we want to have two executors and every executors should run two tasks.

```
topology.setBolt("green", new GreenBolt(), 2).setNumTasks(2)
```

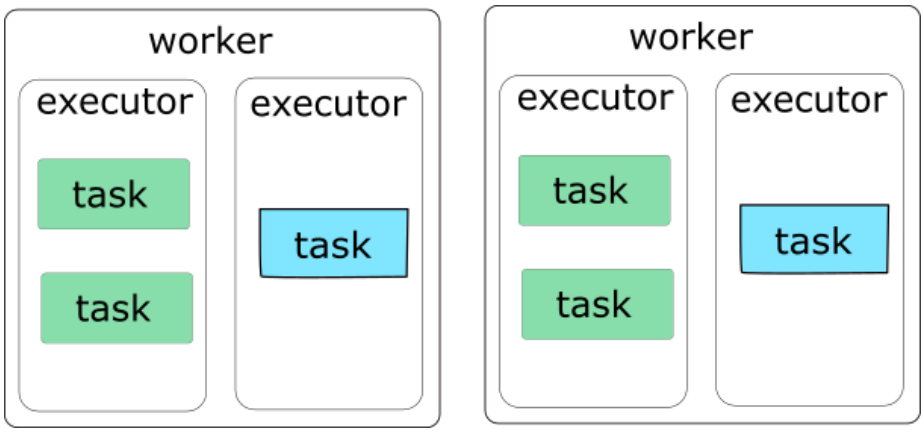


Figure 2.2: Parallelism Example

2.2.4 Stream Groupings

The connection between Spouts and Bolts in Storm is realized by defining the stream groupings. As noted above, for every Bolt multiple instances can be created. Sending the specific parts of the stream to the appropriate instances of the Bolt is realized by using the stream groupings. Storm provides several predefined stream groupings and those are:

1. **Shuffle Grouping** - randomly distribute tuples across the instances of the Bolt such that every instance will receive an equal number of tuples
2. **Fields Grouping** - partition the stream based on one or more fields specified in the grouping. This means that all of the tuples with the same value for the specified field(s) will end up on the same instance
3. **Partial Key Grouping** - same as **Fields Grouping** but additionally the tuples are distributed in a balanced manner across the instances
4. **All Grouping** - the stream is replicated across all of the instances of the Bolt
5. **Direct Grouping** - the producer of the tuples decides which instance of the subscriber will receive the tuple

Additionally, Storm provides the possibility for custom groupings by implementing the **CustomStreamGrouping** interface.

2.2.5 Windows

Working with potentially never-ending data streams is one of the obstacles that need to be overcome in data stream processing. Storing unbounded data streams is impossible since at some point in time any machine will run out of memory. Additionally, having an enormous input set means that all of the processing will take a substantial amount of time to provide the needed results. For that reason, in frameworks like Apache Storm the concept of *windows* exists. This

means that instead of performing the processing logic on the whole input, every Bolt will operate only on the portion of the data defined by the window. Two types of *windows* exist, Sliding Windows and Tumbling Windows. Additionally, the tuples that will be contained in the window can be defined based on time or count. In a **Sliding Window**, the window slides across the data stream based on a specified interval. An example of a time-based sliding window is shown in Figure 2.3 where the window is with length ten seconds and a sliding interval of five seconds. The sliding window is characterized by the fact that the windows can contain overlapping elements demonstrated by the windows *W1* and *W2*. A **Tumbling Window** groups all of the tuples for a specified time or count.

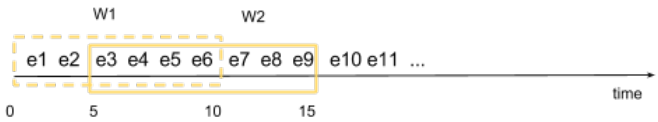


Figure 2.3: Sliding Window Example, from [2]

An example of a time-based tumbling window is shown in Figure 2.4 where the window interval is five seconds. Contrary to the sliding window, in the tumbling window every tuple belongs to one and only one window.

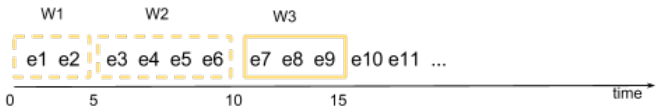


Figure 2.4: Tumbling Window Example, from [2]

2.3 Association Analysis

The problem of finding associations has been studied and covered in many kinds of research and is typically referred to as Association Analysis [22]. Through Association Analysis interesting relationships that are hidden in huge datasets can be found. It is especially useful in market basket data, where the goal is to help retailers find connections between the customer purchases so that they can identify new opportunities for cross-selling products to their customers [8]. Association analysis is applicable in many other domains, such as Web mining, scientific data analysis, medical diagnosis, and others [22].

Let $T = \{t_1, t_2, .., t_N\}$ be the set of all transactions and $I = \{i_1, i_2, .., i_d\}$ be the set of all items. An itemset represents a group of zero or more items and a transaction t_i contains an itemset X if X is subset of the transaction t_i. The support (σ) of an itemset X is the number of transactions that contain the itemset X [22]. From Table 2.1 one example itemset can be $\{Bread, Milk\}$ and this itemset has support two since it occurs in the transactions 1 and 5.

Association analysis is performed by finding Association rules. Association rule represents an implication of the form $X \rightarrow Y$, where X and Y are disjoint item-

Transaction ID	Bread	Beer	Milk
1	1	0	1
2	1	1	0
3	0	0	1
4	0	1	1
5	1	1	1

Table 2.1: Example Market Basket Data

sets i.e., $X \cap Y = \emptyset$. With the use of the Support (Equation 2.1) and Confidence (Equation 2.2) the strength of an Association Rule can be determined [22].

$$supp(X \rightarrow Y) = \frac{\sigma(X \cup Y)}{N} \tag{2.1}$$

$$conf(X \rightarrow Y) = \frac{\sigma(X \cup Y)}{\sigma(X)} \tag{2.2}$$

These metrics help eliminate the Association Rules that occur simply by chance and are actually not interesting from a business perspective.

Association Rule Mining is the problem of finding all association rules with $supp \geq minsup$ and $conf \geq minconf$ where $minsup$ is the threshold for the support and $minconf$ is the threshold for the confidence. The itemsets that satisfy the $minsup$ threshold are called **frequent itemsets** [22]. The **naive** approach for Association Rule Mining is to find every possible itemset and compute the support and the confidence. Immediately it can be concluded that the **naive** approach is not feasible since it has enormous time complexity. The **Apriori Principle** is a more efficient approach, that is one of the widely accepted and used algorithms for Association Rule Mining [19], [23], [22]. Even though several variations of the algorithm exist, they are all based on the same idea: *if an itemset is frequent then all of its subsets are also frequent.* The opposite also holds, *if an itemset is infrequent then all of its supersets are also infrequent.* This rule is possible based on the **anti-monotone** property of the support, which tells us that the support of an itemset will never be greater than the support of its subsets.

Through this rule, many infrequent itemsets are immediately pruned and the rule is the main reason why the algorithm achieves an acceptable execution time in practice. In Figure 2.5 an example of the pruning in the apriori algorithm is shown. If it is determined that the itemset $\{a\}$ is an infrequent itemset, then immediately all of its supersets are ignored in the subsequent iterations.

2.4 Frequent Pattern Trees

Frequent Pattern Trees (FPTrees), firstly introduced by J. Han et al. [13], represent an innovative approach for storing compact and important information about frequent patterns. Through FPTrees, mining for associations can be performed in a faster and more efficient way compared to the other association

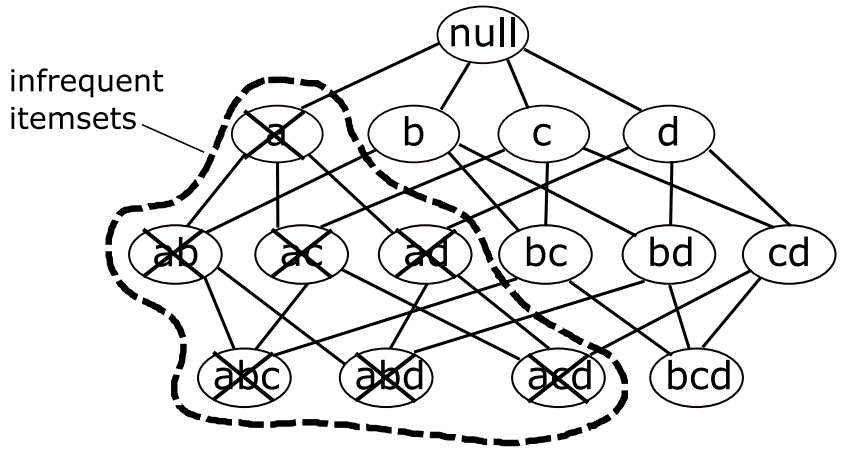


Figure 2.5: Pruning in Apriori Algorithm

mining techniques. As mentioned, most of the mining techniques are based on an Apriori-like approach whose main idea is: if any length k pattern is not frequent in the database then its length $(k + 1)$ super-pattern can also never be frequent. Even though the Apriori-based algorithms achieve good performances, in situations with numerous frequent patterns or long patterns, they have suboptimal performances. The problems are the large number of candidate sets that need to be generated for the frequent itemsets and the continuous, repeated database scans. FPTree and the corresponding mining algorithm (FPGrowth) tackle the disadvantages of the Apriori-based approaches.

Building an FPTree can drastically reduce the size of the database and lower the number of database accesses. The FPTree is built based on the following observations:

1. For frequent pattern mining, only the frequent items are needed. For that reason, one DB scan is needed to determine the set of frequent items.

2. By storing the set of frequent items in a compact structure, the need for repeated scans of the DB is eliminated.

3. If the frequent items are sorted by a fixed order, the transactions that share frequent itemsets can be merged together and the information that multiple transactions share the same frequent items can be represented through a count variable.

4. If the frequent items are sorted in a descending order based on their frequency, more prefixes will be shared by the transactions and the transactions can be merged together. The number of merged transactions can be represented through the count variable.

Based on these observations the FPTree can be constructed in the following way. One DB scan is needed to gather the frequent items and order them in descending order based on their frequency. Once the ordering is done, second scan of the DB is performed to gather the transactions and add them as branches to the FPTree. The frequent items from every transaction are ordered based on

the fixed ordering from the first DB scan. Additionally, a helper table is built, that stores information about the nodes that have the same label.

Example. Let d, b, c, a be the fixed ordering of the frequent items and $< d, c, a, m, p, l >$ be the next transaction from the database. Since, m, p and l are not frequent items we eliminate them from the transaction and as a result, the frequent items $< d, c, a >$ need to be added as a branch in the tree. Since this is the first transaction, the FPTree consists of only the root node which is labeled with 'null'. The creation of the branch is carried out by taking the first frequent item 'd' and checking if the root has a child labeled with 'd'. If not, the node $< d, 1 >$ is added as a child of the root, where 1 represents the number of occurrences of the frequent item 'd'. The node 'd' is added in the helper table as the first node labeled with 'd'. Next, the frequent item 'c' needs to be added as a child to the node 'd'. Again since 'd' does not have any child labeled with c, the node $< c, 1 >$ is added as a child to the node $< d, 1 >$ and the node 'c' is also added in the helper table. The same procedure is repeated and the node $< a, 1 >$ is added as child of the node $< c, 1 >$ and also in the helper table. As a result, the frequent itemset is added as a branch of the FPTree. Let $< d, a >$ be the next frequent itemset that needs to be added to the FPTree. First, the frequent item 'd' is selected. Since, the root already has a node labeled with 'd', only the counter of that node is updated, leading to the node $< d, 2 >$. For the next frequent item 'a', a new node $< a, 1 >$ is created and added as child of the node $< d, 2 >$. Additionally, a link is created between the existing node labeled with 'a' and the newly created node labeled with 'a'. From these two transactions we have the FPTree shown in Figure 2.6.

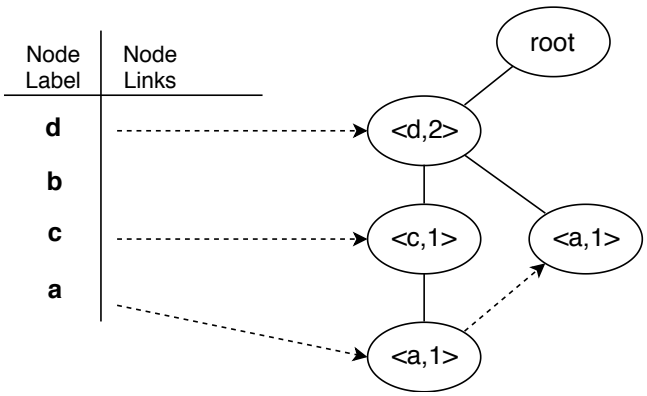


Figure 2.6: Frequent Pattern Tree Example

From the observations and the example, the following definition of an FPTree can be derived. A frequent pattern tree is a tree that

1. Consists of root labeled with 'null', a set of frequent items as nodes of the tree and a helper table.

2. Every node in the tree consists of an item-name (label), count and node link. The item-name represents the name of the frequent item, count represents the number of transactions and node link points to the next node in the tree with the same item name or null if there is no such node.

3. The helper table consists of two fields, node-label which represents the item-name of the node and node-links which points to the first node in the FPTree that has that item name.

The cost of inserting a transaction T in the FPTree is O(|T|), where |T| is the number of frequent items in the transaction T.

The aforementioned algorithm FPGrowth is used to efficiently traverse the FP-Tree and generate the frequent patterns. The FPGrowth algorithm starts with the frequent length-1 pattern (initial suffix pattern), examines only the conditional pattern base of the length-1 pattern (sub-database which consists only of the set of frequent items co-occurring with the suffix pattern), constructs its FP-Tree and performs mining recursively on the tree. In the end, the suffix pattern is concatenated with the new ones from the FPTree. The algorithm avoids the generation of numerous candidate sets and thus achieves better performances from the Apriori-like approaches.

2.5 NoBench Data Generator

Chasseur et al. [10] investigate whether the advantages of JSON documents can be incorporated in traditional relational DBMS whilst maintaining some of the benefits of the relational DBMS. In their work, the authors create an automated mapping layer for storing and querying JSON data in relational DBMS, called **Argo** and they compare their approach with the NoSQL databases. The benchmark used for generating the JSON data and comparing the performances called **NoBench** is one more novelty introduced by Chasseur et al. [10].

NoBench is built following the guidelines of other, already existing database benchmarks. The goal of this benchmark is to identify a compact and straightforward set of queries, that focus on the common operations. NoBench consists of a JSON data generator and twelve elementary queries. The goal of the queries is to simulate operations that are usual for a variety of applications and not to focus on a specific production workload. The NoBench JSON data generator is implemented in Python and it creates JSON objects that contain the following attributes:

- **str1,str2** - unique strings
- **num** - distinct number in the range from 0–N, where N represents the total number of objects that will be generated by calling the NoBench data generator
- **bool** - boolean value that is in 50% of the objects true and in the other 50% false
- **dyn1** - dynamical value that is in 95% of the objects equal to **num** and in the other 5% equal to **str1**
- **dyn2** - dynamical value that is equal to **str1**, **num** or **bool** in 1/3 of the elements

- **nested_arr** - an array object with 0–7 randomly selected strings from the Brown Corpus
- **nested_obj** - an object element with attributes:
  - **str** - string equal to **str1** from different object
  - **num** - number equal to **num** from different object
- **sparse_XXX** - series of attributes from **sparse_000** to **sparse_999**. Every object consists of ten sparse attributes that are chosen from cluster of 1000 attributes.
- **thousandth** - integer value in the range from 0–999 that is actually equal to **num** modulo 1000

Chapter 3

Related Work

Performing joins on data streams in a distributed environment is a topic that has been widely researched and analyzed. Most of the work focuses on performing traditional SQL joins over the tuples defined in a specific window. Even though many of the approaches are scalable, efficient and perform well for large datasets, they differ from the focus of this thesis which is to perform stream joins when the join parameters are unknown. Subproblem of distributed stream joins is to partition the data in an efficient and even manner such that the joining can be performed in a more productive and acceptable way. Without the partitioning, it will be impossible to distribute the data evenly among the different machines in the cluster.

3.1 Distributed Stream Joins

The authors of the paper [15] analyze several stream join algorithms and inform the most suitable approaches for different scenarios. Additionally, they analyze situations where resources and memory are insufficient and give suggestions for more optimal results. Elseidy et al. [12], represent an online join operator that can adjust itself through routing and repartitioning. However, all of the joining algorithms considered are not comparable to the join requirements in this thesis. MapReduce implementations of equi-joins can be found in [4, 7], which again focus on performing traditional, SQL-like joins.

3.1.1 BiStream

BiStream introduced by Lin et al. [18] is an example of a distributed stream join model. The main novelty, referred to in the paper as join-biclique, is organizing the processing units of a cluster as a complete bipartite graph, where every side will correspond to a particular relation. The main advantages of using a join-biclique are that the relations are not replicated, which leads to a more memory efficient approach and the processing units are not connected to one another, which makes the approach more scalable and elastic. For distributed stream joins a two-level architecture is proposed, consisting of a router and a joiner. The job of the router is to distribute the tuples to the appropriate processing unit, in this situation the joiner. The joiner is responsible for performing the actual join of the tuples. As input, the joiner receives two different types of messages: tuples that need to be stored and tuples that need to be joined with the stored tuples. Mainly the *BiStream* approach is developed for dealing with equi-joins, which is achieved by storing the data in the joiners in hash maps.

3.1.2 Approximate Distributed Stream Joins

ApproxJoin, introduce by Quoc et al. [20] is a distributed stream joining approach that provides approximate results. The approach is realized by using a Bloom Filter sketching and stratified sampling through which the statistical properties of the join output are preserved. With the Bloom Filter the network traffic is reduced and with the innovative stratified sampling, the statistical qualities of the join results are preserved. The approach is implemented in Apache Spark and it consists of two phases:

- *Filtering redundant items* - where the items that are going to participate in the join operation are determined and the non-participating items are excluded
- *Approximation in distributed joins* - samples from the dataset are created that fairly represent the datasets. The sampling is combined with stratified sampling for more adequate results.

The same observation applies to this approach, meaning that the parameter based on which the join is performed needs to be known and provided in advance.

D-Stream, introduced by Zaharia et al. [24], is an approach for structuring streaming computations in a more efficient way by splitting the streaming computations in deterministic batch computations into specified time intervals. For every batch defined over a timing interval, a separate MapReduce job is performed. Even though this approach provides a more effective fault-recovery, it is not applicable in our scenario since by grouping the data into small batches, candidate tuple pairs for joining may miss each other. Hence this approach, same as *ApproxJoin*, can only provide approximate join results.

3.1.3 Photon

The authors of [6] introduce *Photon*, distributed stream join approach, mainly focused on joining web search queries and user clicks on advertisements in Google. The main idea of the system is to enable advertisers to improve their campaigns in real-time so they can fulfill the changing user behaviors. The join between the search queries and the user clicks on advertisements is performed by using a unique identifier that is present in both events. Through this identifier, *Photon* can perform stream joins while guaranteeing exactly-once semantics. However, again the same observation can be made, the parameter based on which the join is performed is already known and given.

3.2 Partitioning

Joining massive streams in real-time requires the data to be efficiently distributed and partitioned among the machines. Kernighan-Lin algorithm [16] is one of the best-known algorithms for graph partitioning. The main focus of this algorithm is to minimize the edge-cut which is achieved by moving vertices between partitions. However, in a dynamic environment, this approach is

computationally expensive due to the fact that the data changes as the stream flows and any partition will be valid only for a short period of time. The work presented by the authors of [9,16,21] is based on the Multilevel Graph Partitioning [14]. They try to improve the partitioning by combining it with different heuristics. However, all of the algorithms are centralized algorithms, meaning that they assume random access to the entire graph. When processing data streams in a distributed setting, the access to the entire graph is neither feasible nor achievable.

3.2.1 Tracking Set Correlations at Large Scale

Alvanaki and Michel [5], presented an approach for finding correlations between tags in Twitter tweets. The correlation is computed by calculating the Jaccard Coefficient. For realizing their approach, the authors investigate several partitioning algorithms and inform about their suitability by measuring the load balance, the communication overhead and the replication of tags between partitions. The approach is implemented with the Apache Storm [1] stream processing framework.

Idea and Realization

Finding the correlation between tags is realized by first creating partitions based on the incoming tweets, then performing updates on the created partitions to incorporate the newly arrived tags and additionally, recomputing the partitions when their quality decreases. In Figure 3.1 the topology for the approach is shown.

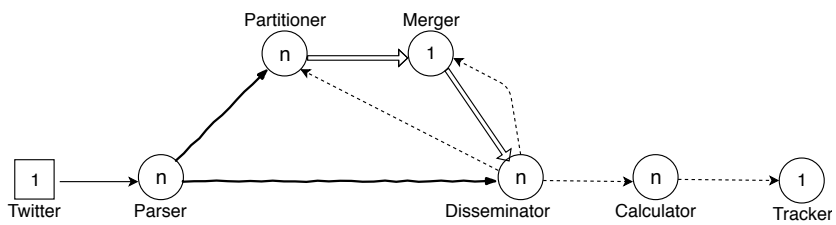


Figure 3.1: Topology for Finding Tag Correlations

The **Parser Bolt** is responsible for parsing the Twitter tweets that are received using *shuffle grouping* from the Spout of the topology. For each tweet, it extracts the hashtags and outputs a tuple of the form $(timestamp, s_i)$ where $timestamp$ is the time of the arrival of the tweet in the system and s_i is the set of tags extracted from the tweet.

The **Partitioner Bolt** is responsible for creating the partitions using one of the partitioning algorithms (SC, DS) explained in Chapter 5. Multiple instances of the *Partitioner Bolt* are created and they receive the tuples from the *Parser Bolt* based on *fields grouping*. The *Partitioner Bolts* maintain a sliding window from the incoming tagsets over which the partitioning is performed.

The **Merger Bolt** creates the global k partitions out of the local partitions that are computed by every *Partitioner Bolt*. Only one instance of the *Merger Bolt* is used. Additionally, if any tagset is not present in the existing partitions, the *Merger* is informed by the *Disseminator* and tries to find the best partition for the recent tagset.

The **Disseminator Bolt** receives the partitions from the *Merger* and creates an index that informs which *Calculator* is responsible for what partition. For every tagset received from the *Parser*, it locates the assigned *Calculators*, by checking to which partitions the tagset belongs, and sends the tagset to the appropriate *Calculators*. Furthermore, for every newly seen tagset that is not present in the computed partitions it informs the *Merger*. Multiple instances of the *Disseminator Bolt* are used.

The **Calculator Bolt** computes the Jaccard coefficient for the tagsets received from the *Disseminator*. It receives the tagsets from the *Disseminator* based on *direct grouping*. For parallelization, multiple instances of the *Calculator Bolt* are used.

The **Tracker Bolt** receives the Jaccard coefficients that are emitted by the *Calculators* and performs further computations. Since, multiple coefficients can be emitted for the same tagset, the job of the *Tracker* is to select and report the correct Jaccard coefficient.

Chapter 4

Approach

The main goal of our work is developing an efficient, acceptably balanced approach for joining JSON documents. Since we are working with data stream characterized by large volume and velocity, we implement our approach in a distributed environment by focusing on scalability and improved performance. For reducing the communication overhead and equally distributing the load among the machines in the cluster, an appropriate partitioning technique is used. *Apache Storm* is the framework used for realizing our approach.

In this section, we will first start by examining approaches based only on Apache Storm without using any kind of partitioning. By analyzing these approaches and their respective drawbacks, we will introduce the need for our approach. Afterward, we will introduce our own distributed approach, describe the architecture and explain in details the constituent components.

4.1 Naive Approaches

In Apache Storm the communication between *Spouts* and *Bolts* is realized by emitting tuples. A tuple represents a list of values, and every value can be of any kind. Having in mind that our ultimate goal is joining all the documents based on our join definition presented in Section 1.1, a first approach can be developed where every document will be emitted by the whole set of key-value pairs. This means that all of the documents that have an exactly equal set of key-value pairs will always end up on the same machine, thus, eliminating the need for any partitioning algorithm for distributing the data.

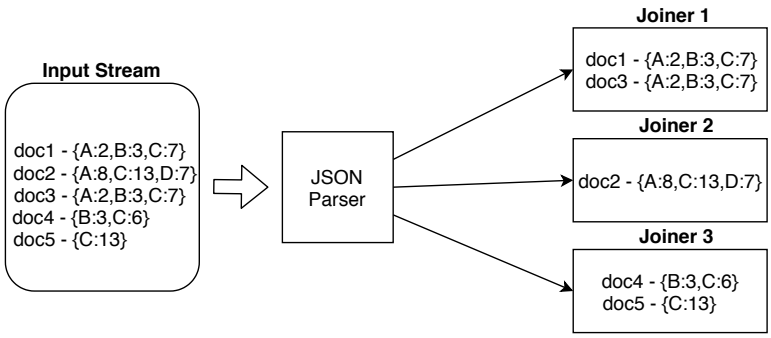


Figure 4.1: Naive Approach 1

In Figure 4.1, an example for sending documents based on the whole set of key-value pairs is presented. The JSON Parser represents the *Spout* responsible for accepting the documents, parsing them and sending the documents to the

appropriate machines. As shown, every document that has the exact same set of key-value pairs will always end up on the same machine. If we have a closer look at our definition for join, we can see that two documents are joinable if they share at least **one** common key-value pair. By emitting the documents in this way, it is not possible to satisfy our join definition since documents that have common key-value pairs will end up on different machines. For example in Figure 4.1, *doc2* and *doc5* are on different machines even though they are join partners based on our definition for join. As a result, with this approach, we do not guarantee that all of the joinable documents will be provided as an outcome and for that reason, we consider this to be our first naive approach.

From the first naive approach, we can conclude that there is a need for an approach that will provide the complete set of joinable documents. By analyzing the drawbacks and drawing conclusions, an approach where the documents will be emitted based on single key-value pairs can be considered. With this approach, the requirements of our join definition will be satisfied since all of the possibly joinable documents will end up on the same machine. An additional implementation for disregarding the documents that have conflicting values for the same key is required, but in the end, the complete and correct set of joinable documents will be provided.

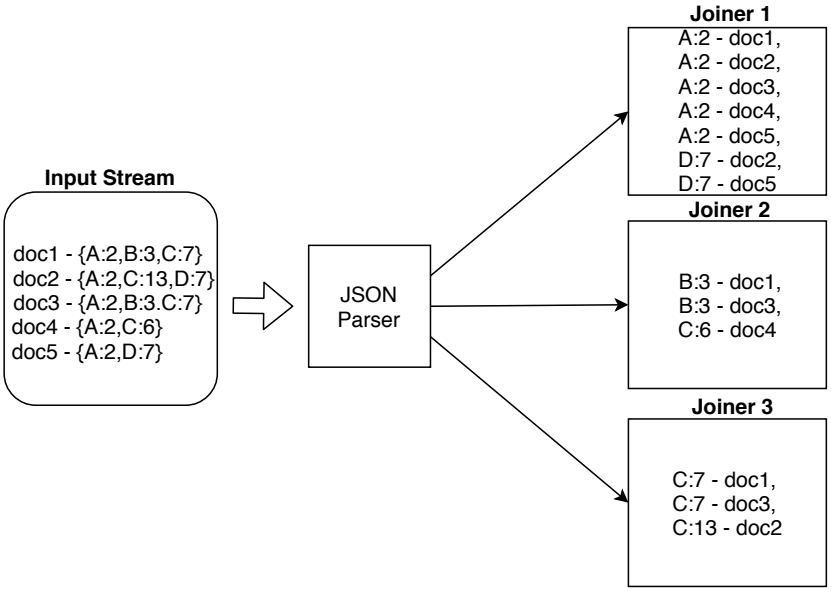


Figure 4.2: Naive Approach 2

In Figure 4.2 an example for sending the documents based on single key-value pairs is provided. One can observe that all of the possibly joinable documents will always be on the same machine. For example, all the documents that can be joined by the key-value pair *B:3* are located on machine 2. On the other hand, difference in the number of documents per machine can be noticed, where machine 1 has more documents compared to the other machines. This is a result of the unequal number of occurrences of the key-value pairs in the documents. If one key-value pair is extremely common and it occurs in 90 percent of the

documents, this will result in one machine processing 90 percent of the whole set of documents. This means that firstly, the machines will have unequal load and secondly the machine that has the most common key-value pair will represent a bottleneck affecting the execution time of the whole approach. Furthermore, by emitting the documents based on single key-value pairs, the communication overhead will be drastically increased. In the worst case, every document will be emitted to a different machine for every key-value that it has. For a large set of documents, the communication overhead will be enormous, condemning the whole approach as not executable. Because of the aforementioned issues, this approach is considered to be our second naive approach.

From the two naive approaches, we can conclude that the documents should be emitted based on single key-value pairs which will guarantee that all of the joinable documents will be provided as output. The drawbacks of the second naive approach emphasize the need for an efficient partitioning algorithm that will effectively distribute the documents across the machines providing acceptable communication overhead. Additionally, the documents will be partitioned across the machines in a way that every machine will handle an approximately equal number of documents, leading to a good load distribution across the machines. In the following section, the architecture and the realization of our proposed approach will be presented.

4.2 Topology for Joining JSON Documents

In order to satisfy the requirements for a scalable approach, that can provide acceptable load balancing and communication overhead, we propose the topology represented in Figure 4.3.

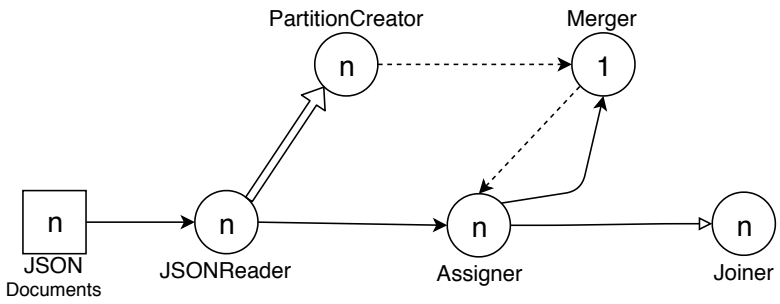


Figure 4.3: Join Approach Topology

Mainly, the idea for our topology is based on the topology presented by Alvanaki and Michel [5]. Although, the *PartitionCreator* bolt and the *Merger* bolt carry similar names as the bolts introduced by Alvanaki and Michel [5], the algorithms used for partitioning are different. The actual implementation of the algorithms will be further discussed in the following subsections and in Chapter 5. The *Assigner* bolt corresponds to the *Disseminator* bolt, performing the same work of delivering the tuples to the appropriate partition. Since the main goal of Alvanaki and Michel [5] is finding correlations between Twitter tags, the

Calculator bolt is replaced with our *Joiner* bolt. The main focus of the *Joiner* bolt is to determine the join partners for a particular count-based window. In the following sections, a detailed explanation of the integral components of our topology is provided.

JSONReader

The *JSONReader* spout produces the stream of JSON documents that will be handled by our approach. Since we are working with JSON documents generated through the use of the NoBench benchmark [10], the first task of the *JSONReader* spout is to call the script for generating the JSON documents. The script as a result of the call creates an array of JSON objects. We consider every JSON object to be a separate JSON document. The second task of the spout is to retrieve and parse the output from the script. Since the JSON objects do not have a name, we store a counter at every instance of the spout, that is updated for every new JSON object that is processed. For every JSON document, a unique name is created by concatenating the id of the spout and the counter that is stored locally at the spout. The next task of the spout is to delegate the documents to the subscribed bolts. As an output, the *JSONReader* emits a tuple of the the following form $(kv_i, d_i, timestamp)$, where kv_i represents the set of key-value pairs for the document d_i and $timestamp$ is the time of the arrival of the document d_i in the system. The letter **n** inside the circle representing the *JSONReader* in Figure 4.3 indicates that multiple instances of the spout can be created.

When calling the NoBench script, one can specify the number of JSON objects that should be generated. We make use of this characteristic to restrict the number of documents that will be processed by every spout and with that provide a realistic flow of documents that will be analyzed by the topology.

PartitionCreator

The *PartitionCreator* bolt is responsible for creating the partitions for the JSON documents based on one of the algorithms explained in Chapter 5. The *PartitionCreator* is subscribed to the *JSONReader* using *partial key grouping* on the set of key-value pairs kv_i. Through this type of grouping, the documents that have an equal set of key-value pairs will be always forwarded to the same instance of the *PartitionCreator*. To enable faster execution of the partitioning algorithms, multiple instances of this bolt can be created.

For creating the partitions, the *PartitionCreator* maintains a sliding window over the incoming documents. As explained in Section 2.2.5, two types of sliding windows can be used, count-based or time-based sliding window. For our work we have chosen the count-based sliding window, which means that whenever the *PartitionCreator* receives X documents, the partitioning is triggered and the appropriate partitioning algorithm is executed. After the creation of the partitions, the *PartitionCreators* emit tuples of the following form $(\{pr_1, pr_2, .., pr_k\})$ where pr_i represents a partition consisting of a set of key-value pairs kv_i.

Once the first partitioning is performed, the *PartitionCreator* continues to maintain a set of documents of size X. One of our design decisions is that we would always like to have the most recent X documents, so that whenever the repartitioning is triggered it would always be performed on the most up to date documents. For storing the most recent documents, when a new document arrives, the document with the oldest timestamp is replaced with the newly arrived document.

Merger

Having multiple *PartitionCreator* bolts means that the number of created partitions will exceed k, the required number of partitions. This imposes the need for an additional component, the *Merger* bolt, that will use the local partitions created by every *PartitionCreator* and construct the final k partitions. Mainly, the *Merger* executes a part of the same partitioning algorithm used by the *PartitionCreator*. Therefore, the *Merger* can be seen as another *PartitionCreator* that receives as input $n * k$ partitions, where n represents the number of *PartitionCreator* instances, and it creates the final k partitions. Once the final k partitions are created, the *Merger* informs the *Assigner* bolts by emitting a tuple of the following form $(\{pr_1, pr_2, .., pr_k\})$.

Compared to the other components, there is only one instance of the *Merger* bolt. This is a mandatory requirement since we need a consistent set of k partitions, which will be used by the *Assigners* for directing the documents to the correct *Joiners*.

Another responsibility of the *Merger* is to find the appropriate partition for documents whose key-value pairs are not present in any of the existing partitions. The *Merger* actively waits for messages from the *Assigners* about documents with unseen key-value pairs. For any such document, delivered by the *Assigner*, the *Merger* executes a part of the appropriate partitioning algorithm, finds the best partition and informs all the *Assigners* for the updated partitions. The updating of the partitions is explained in detail in Section 4.3.

Assigner

The *Assigner* bolt is responsible for directing the documents to the appropriate *Joiner* bolts. For performing the allocation of the documents, the *Assigner* requires the final k partitions. For that reason, the *Assigner* is subscribed to the *Merger*. From the received partitions, the *Assigner* creates an index from the key-value pairs to the *Joiners*. The index indicates to which *Joiners* a document should be sent, based on the key-value pairs. Additionally, the *Assigner* is subscribed to the *JSONReader* using shuffle grouping and for every incoming document, it uses the index to determine the *Joiners* to which the document should be sent. For every received document from the *JSONReader*, the *Assigner* sends a tuple of the form (kv_i, d_i) to the appropriate *Joiners*. For example, let $doc_1 = \{a : 2, b : 3, c : 4\}$ be the next document received by the Assigner and Joiner 1 be responsible for the key-value pairs $\{a : 2, b : 3, c : 4\}$ and Joiner 2 be responsible for the key-value pairs $\{a : 2, b : 3\}$. The Assigner

will check the index, conclude that for the key-value pair $a : 2$ the Joiners 1 and 2 are assigned and emit the tuple $(\{a : 2, b : 3, c : 4\}, doc_1)$ to both Joiner 1 and Joiner 2 using *direct grouping*. To reduce the communication overhead we create a *blacklist* indicating which document was already sent to which of the *Joiners*. From the example, once doc_1 was emitted to Joiner 1 for the key-value pair $a : 2$, that information will be stored in the blacklist and doc_1 will not be emitted again for the key-value pairs $b : 3$ and $c : 4$. To accelerate the communication with the *Joiners*, multiple instances of the *Assigner* bolt can be created.

Working with a sliding-window over the documents means that there can be documents whose key-value pairs are not present in the created partitions. This means that the *Assigner* has a second responsibility, apart from directing the documents to the *Joiners*, which is informing the *Merger* for the previously unseen key-value pairs. Once the *Assigner* receives a document that has at least one key-value pair that is not present in the existing partitions, it sends a tuple of the form (kv_i, doc_i) to the *Merger*. If the documents are constantly changing in terms of their key-value pairs, by sending a tuple for every new document to the *Merger*, we can create a bottleneck and with that disrupt our whole system. In our approach, we assume that the percentage of unseen documents will be acceptable, making it possible for the *Merger* to handle the number of incoming tuples. A possible solution for the case where the change rate of the documents is unacceptably high is presented in Section 8.1.

Joiner

Joiners are responsible for performing the actual joins on the assigned documents. They are registered to the *Assigners* and receive as input documents on which they should perform the appropriate join algorithm. The explanation of the used join algorithms is given in Chapter 6. Since the *Joiners* are unfamiliar with the complete set of documents, they perform the join algorithm only on the assigned set of documents. This oblivious behavior of the *Joiners* directly contradicts with our requirement for generating all join partners for a given window. To overcome this problem, once the join partners are computed, the *Joiners* create a CSV file where for every document all of the join partners are listed. Generating CSV files creates the possibility for developing an application that can extend our approach and by collecting all CSV files that are produced, construct the whole set of join partners for the particular sliding-window. For example, let document doc_1 be joinable with both doc_2 and doc_3 and through the partitioning let doc_1 and doc_2 be assigned to Joiner 1 and doc_1 and doc_3 be assigned to Joiner 2. By processing the CSV files generated from Joiner 1 and Joiner 2, we can gather the information that doc_1 is joinable with both doc_2 and doc_3.

The *Joiners* execute the join algorithms every x time units. This is accomplished by using *tick tuples* in Apache Storm. Tick tuples are useful when one needs to perform the same logic repeatedly on a schedule. A tick tuple is received every x time units by the *Joiners* which triggers the execution of the join approach and consequently the storing of the generated join partners in a CSV file.

4.3 Updating Partitions

Handling the documents whose key-value pairs have not been seen previously is of great importance since otherwise, the Assigners will not know to which Joiner to emit the document and with that documents may be lost. As a result, the whole approach can provide inaccurate join partners. The Assigners are the main component where the triggering of the update occurs since they connect the partitions with the Joiners. When the Assigners receive a document, they immediately notice when a key-value pair is not present in the partitions since they cannot find the Joiner assigned for that key-value pair. The Assigners inform the Merger about the unseen key-value pair by sending the whole document from where the key-value pair was detected. When a document is sent to the Merger for an update, the Assigners store that document so that it can be sent to the correct Joiner once the revised partitions are created. The job of the Merger is to determine the best partition to which the document should be assigned. Depending on the partitioning algorithm used, the Merger uses a different approach. The assignment of an unseen document for every partitioning strategy is described in Chapter 5. Instead of performing a single update for every document received from the Assigner, the Merger stores the documents locally and when a time trigger is generated, implemented through a tick tuple, it finds the most appropriate partitions for all of the currently stored documents. After the partitions are updated, the Merger informs all the Assigners. The Assigners use the revised partitions to update their local index and assign the Joiners to the appropriate key-value pairs.

Chapter 5

Partitioning Approaches

Performing joins over data streams requires a distributed approach, where the task of join processing will be executed in parallel on multiple machines and every machine will be responsible for finding the joining partners in the allocated set of documents. To enable an even distribution of the documents with acceptable communication overhead and replication factor, an efficient and scalable partitioning algorithm is required. Without an appropriate partitioning approach, one may end up in a scenario like the one described in the second naive approach in Section 4.1, where one of the machines will be responsible for processing the majority of the documents while the other machines stay idle. This will result in a bottleneck which will drastically affect the system, leading to a situation where all of the benefits of the distributed processing are lost.

As a starting point, we have investigated the proposed partitioning algorithms by Alvanaki and Michel [5], mainly the *Disjoint Sets Algorithm* and the *Set-Cover–Based Algorithms*. These partitioning algorithms, model the problem as a graph partitioning problem, where every vertex represents a set of tags and an edge exists between two vertices if they have common tags [5]. Since they have investigated the applicability of the approaches on Twitter tweets that are characterized by having less text and with that fewer tags in the text, we wanted to examine whether the same approaches will be relevant when working with JSON documents, where every JSON document contains a higher number of key-value pairs as compared to the Twitter tweets.

By analyzing and constantly working with JSON documents, we inferred that there is a possibility for an alternative partitioning algorithm. The previous partitioning algorithms created the partitions by determining which documents (in the case of [5] tweets) have common elements. We saw that there is a possibility to work from a different perspective, by analyzing the connections between the key-value pairs within the documents. We identified that within the JSON documents some key-value pairs always occur together, meaning that if the one key-value appears then always the other key-value pair appears (equivalence relationship) and on the other hand, some key-value pairs appeared only when a specific key-value pair appeared and not alone (implies relationship). Based on these conclusions, a partitioning approach by utilizing the *Association Rule Mining* principles was created and through the equivalence and implies relationships between the elements of the JSON documents, the key-value pairs were grouped into partitions.

In the following sections, the actual implementation of all the partitioning algorithms is explained. For the partitioning algorithms that were implemented in the distributed setting, we explain how they were modified so that they can be used in the distributed scenario and how this modified partitioning algorithms are realized by the Storm topology explained in Section 4.2.

5.1 Disjoint Sets

The partitioning algorithm based on *Disjoint Sets* strives to find all of the disjoint sets from the given set of document. Disjoint sets are formed by grouping all the documents that share at least one key-value pair. This algorithm guarantees that all the key-value pairs from one document will be present within one disjoint set and additionally all the documents that are candidates for join will always be in the same partition. Hence, every document will be sent to one and only one machine, leading to optimal communication overhead and replication factor.

Algorithm 1 Disjoint Sets Algorithm (**DS**)

Input: Integer k, Set S_KV of kv-pairs sets s_kv_i, Set of kv-pairs KV, Set of documents D

Output: Set PR of k partitions pr_i

1: $DS = \{\}, PR = \{\}$
2: **while** $KV \neq \emptyset$ **do**
3: **find** $ds_j = \cup_i s_kv_i$ disjoint set of kv-pairs
4: $l_j = |\cup_i D_i|, d_i \in ds_j$
5: $DS = DS \cup \{ds_j\}$
6: $KV = KV \setminus ds_j$
7: **end while**
8: **while** $DS \neq \emptyset$ **do**
9: $ds_i = argmax_{\mathrm{ds}_j} l_j$
10: $DS = DS \setminus ds_i$
11: **if** $k > 0$ **then**
12: $pr_k = ds_i$
13: $PR = PR \cup \{pr_k\}$
14: $k = k - 1$
15: **else**
16: $pr_i = argmin_{\mathrm{pr}_j} \sum_{ds_k \in pr_j} l_k$
17: $pr_i = pr_i \cup ds_i$
18: **end if**
19: **end while**
20: **return** PR

Algorithm 1 provides the realization of the *Disjoint Sets* partitioning algorithm. The algorithm starts by computing all of the disjoint sets from the given set of key-value pairs and for every disjoint set calculates the load (Algorithm 1, Lines 2–7). The disjoint sets are computed by grouping together all the documents whose sets of key-value pairs have at least one common key-value pair. The load of a disjoint set is calculated by counting the number of documents in which every key-value pair of the disjoint set appears. Once the final disjoint sets are computed, they need to be assigned to the available k partitions. The algorithm iterates over the disjoint sets and as next to consider the disjoint set with the largest load (ds_i) is chosen (Algorithm 1, Line 9). If there are remaining partitions without any elements then the disjoint set d_i is added in the next free partition and the number of free partitions is reduced (Algorithm

1, Lines 11–14). If all the partitions have been filed, the disjoint set ds_i is added to the partition that has the minimal load (Algorithm 1, Lines 14–18). The same procedure is repeated until all the disjoint sets have been assigned to partitions.

The Disjoint Sets algorithm ensures that no key-value pair will be replicated in multiple partitions. This means that at the end every document will be sent to one and only one machine since the partitions will not have overlapping elements. This algorithm guarantees to send all of the joinable documents to the same machine and with that, it guarantees that no join will be missed. As a result, this algorithm creates the best possible replication of the documents across the machines.

The problems of this approach come from the input set of documents. If the documents form groups, meaning that the key-value pairs appear in only one smaller group of documents and not in others then this approach will be able to find disjoint sets that are balanced in terms of the load. On the other hand, if the documents that we are working with are all about the same topic then many of the key-value pairs will be shared and almost all of the documents will be connected to one another either directly (through the same key-value pairs) or through links (doc1 is connected to doc2 and doc2 is connected to doc3 making them part of the same disjoint set even though doc1 and doc3 do no share the same key-value pairs). The disjoint sets found for these documents will be very poorly balanced. It can happen that one disjoint set is responsible for 95% of the documents and all of the other disjoint sets for 5% of the documents. This means that one machine will represent our bottleneck and will impact the execution time of the whole system. It can also happen that the number of disjoint sets is smaller than the number of partitions that we want to create, resulting in the fact that many machines will stay idle.

In order to avoid these problems, the Disjoint Sets algorithm can be combined with other algorithms such as the Graph Cut Algorithms which will split the large disjoint sets into smaller ones. As an example the Disjoint Set algorithm can be combined with the Kerninghan-Lin graph cut algorithm that tries to split the graph by trying to achieve a minimal cut in the graph in terms of the weights of the edges. By combining the DS algorithm with another algorithm additional complexity is introduced and even though the partitioning problem will be solved, the execution time of the algorithm may be unacceptable.

5.2 Set-Cover–Based Algorithms

The *Set-Cover–Based Algorithms* give us another possibility for partitioning the set of documents. Even though, Alvanaki and Michel [5] come to the conclusion that the Disjoint Sets algorithm is more suitable for finding partitions, the data used in their paper is different than our data. Tweets usually do not appear in combination with tweets from different topics, whereas in our work all the documents may be about the same topic and grouping them into disconnected groups may not be possible. That is why the Disjoint Sets algorithm will form unbalanced partitions in terms of load and they can also be less in number than

the actual number of partitions that we would like to create. For that reason, the Set-Cover–based algorithms can be feasible for partitioning the documents and they can also be used in combination with the Disjoint Sets algorithm in order to solve the problem with load balancing.

In its original form, the Set Cover Problem (SCP) describes the task, given a set of sets S find the smallest set of subsets that cover all of the elements from the set of all sets S. For example if $S = \{\{a : 1, b : 2, c : 3\}, \{b : 2, d : 4\}, \{c : 3, d : 4\}, \{d : 4, e : 5\}\}$ then the set S can be represented with less sets while at the same time covering all elements from S through the subsets $\{\{a : 1, b : 2, c : 3\}, \{d : 4, e : 5\}\}$. The original Set Cover Problem is not feasible for our approach since we would like to have all of the key-value pairs that appear in one document together in the same partition which will reduce the communication overhead and with that the execution time of the whole approach.

To make the Set Cover Problem usable in our scenario, restriction on the SCP problem is imposed, which specifies that all sets of key-value pairs from the documents need to be present in the result set. For creating the initial k partitions a modification of the Budgeted Maximum Coverage Problem (BMCP) [17] is used and once the initial partitions are computed the remaining sets of key-value pairs are assigned to the existing partitions by focusing on either the optimization of the processing load or the communication overhead. The explanation of the Budgeted Maximum Coverage Problem in our scenario is given by the following example. Let there be collection of sets $S = \{s_1, s_2, ..., s_n\}$ where every set consists of elements $KV = \{kv_1, kv_2, ..., kv_m\}$. Every set s_i is associated with cost c_i and every elements has weight w_i. The goal of the BMCP approach is to find sets $S^{'}$ whose total cost does not exceed the budget B while maximizing the total weight of the covered elements. In this thesis, the key-value pairs represent the elements and our sets are the sets of key-value pairs for every document. Since all the key-value pairs have equal importance there is no weight assigned to them. This means that the focus will be on finding sets of key-value pairs that have minimal cost and maximal coverage.

Algorithm 2 presents the approach for creating the initial k partitions, which will be used at a later point for finding the final k partitions. In every iteration, the set of key-value pairs that has the minimal cost and the maximal number

Algorithm 2 Set Cover Based Algorithms - Creation of Initial Partitions

Input: Integer k, Set S_KV of kv-pairs sets s_kv_i, Set of documents D
Output: Set PR of k partitions pr_i

1: $CoveredKV = \{\}, PR = \{\}$
2: **while** $k > 0$ **and** $S_KV \neq \emptyset$ **do**
3: $\quad s_kv_i = argmin_{\mathrm{s_kv}_j} c_j$ **and** $argmax_{\mathrm{s_kv}_j} |s_kv_j \backslash CoveredKV|$
4: $\quad pr_i = s_kv_i$
5: $\quad PR = PR \cup \{pr_i\}$
6: $\quad S_KV = S_KV \setminus \{s_kv_i\}$
7: $\quad CoveredKV = CoveredKV \cup s_kv_i$
8: $\quad k = k - 1$
9: **end whilereturn** PR

of key-value pairs that have not been covered is selected (Algorithm 2, Line 3) and used to fill the next free partition (Algorithm 2, Lines 4–5). Once the set of key-value pairs is added to the final partitions, the set of covered key-value pairs $CoveredKV$ is updated with the elements from the current set (Algorithm 2, Line 7). The algorithm iterates until all k partitions are filed with elements.

The cost differs depending on whether the optimization objective is the communication overhead or the processing load. If the focus is the optimization of the **communication overhead** then the cost c_i for the set s_kv_i is the number of key-value pairs that are already assigned to the set of covered key-value pairs $CoveredKV$. If the focus is the optimization of the **processing load** then the cost c_i of the set s_kv_i is the difference of the load of the set compared to the optimal share of the load. Every set s_kv_i consists of key-value pairs kv_i. The load for a key-value pair kv_i is the number of documents in which the key-value pair appears. The load for a set s_i is the cardinality of the union of all of the documents in which the key-value pairs of the set appear. In the mth iteration of the algorithm, $CoveredKV$ contains $m-1$ sets and the mth set is about to be selected. The optimal distribution of the load in the mth iteration is $pl_{\text{op}} = \frac{1}{m}$, meaning that the load is equal on all machines. The real share of the load for the set s_kv_i is $pl_n = \frac{l_n}{\sum_i^{m-1} l_i + l_n}$. As a result, the cost c_i for the set s_kv_i is defined as $|pl_{\text{op}} - pl_n|$.

The next step is to create the final k partitions based on the initial partitions. To achieve this, the remaining sets of key-value pairs need to be assigned to one of the initial partitions. Selecting the next set of key-value pairs and choosing the partition to which the selected set will be assigned depends on the optimization objective. Two variants for assigning the remaining sets are created, one focused on optimizing the communication overhead and another focused on optimizing the processing load.

Algorithm 3 Set Cover Based Algorithms - Phase 2: Assigning the Remaining Tagsets by Optimizing the Communication Overhead (**SCC**)

Input: Set PR of k initial partitions pr_i, Set S_KV of kv-pairs sets s_kv_i, Set of documents D
Output: Set PR of k final partitions pr_i
1: $CoveredKV = \cup_i pr_i$
2: **while** $S_KV \neq \emptyset$ **do**
3: $s_kv_i = argmin_{\text{s_kv}_j} |s_kv_j|$ **and** $argmax_{\text{s_kv}_j} |s_kv_j \backslash CoveredKV|$
4: $pr_i = argmax_{\text{pr}_j} |s_kv_j \cap pr_j|$ **and** $argmin_{\text{pr}_j} \sum_{s_kv_j \in pr_j} l_k$
5: $pr_i = pr_i \cup s_kv_i$
6: $S_KV = S_KV \setminus \{s_kv_i\}$
7: $CoveredKV = CoveredKV \cup s_kv_i$
8: **end whilereturn** PR

Algorithm 3 outlines the assignment of the remaining sets of key-value pairs to the initial k partitions when optimizing for communication overhead. As the next set of key-value pairs s_kv_i, the set that has the most uncovered key-value pairs and the least number of key-value pairs is selected (Algorithm 3, Line 3).

The partition pr_i to which the selected set s_kv_i will be added is chosen by selecting the partition that has the most elements in common with the selected set and at the same time the least load (Algorithm 3, Line 4). Once the partition is updated (Algorithm 3, Line 5), the key-value pairs from the selected set are added to the set of all covered key-value pairs $CoveredKV$ (Algorithm 3, Line 7) and the algorithm continues until all the sets have been assigned to partitions.

Algorithm 4 Set Cover Based Algorithms - Phase 2: Assigning the Remaining Tagsets by Optimizing the Processing Load (**SCL**)

Input: Set PR of k initial partitions pr_i, Set S_KV of kv-pairs sets s_kv_i, Set of documents D
Output: Set PR of k final partitions pr_i
1: $CoveredKV = \cup_i pr_i$
2: **while** $S_KV \neq \emptyset$ **do**
3: $s_kv_i = argmin_{\mathrm{s_kv}_j} |s_kv_j \cap CoveredKV|$ **and** $argmax_{\mathrm{s_kv}_j} l_j$
4: $pr_i = argmax_{\mathrm{pr}_j} |s_kv_j \cap pr_j|$ **and** $argmin_{\mathrm{pr}_j} \sum_{s_kv_j \in pr_j} l_k$
5: $pr_i = pr_i \cup s_kv_i$
6: $S_KV = S_KV \setminus \{s_kv_i\}$
7: $CoveredKV = CoveredKV \cup s_kv_i$
8: **end whilereturn** PR

Algorithm 4 represents the assignment of the remaining sets of key-value pairs to the initial k partitions when the optimization objective is the processing load. The next set s_kv_i that will be considered is chosen by selecting the set that has the maximal load and the minimal number of covered key-value pairs (Algorithm 4, Line 3). The partition pr_i to which this set will be added is chosen by determining the partition that has the minimal load and the maximal number of elements in common with the selected set (Algorithm 4, Line 4). The covered key-value pairs are updated with the elements from the selected set (Algorithm 4, Line 7) and the algorithm is repeated until a partition has been found for all the sets.

With the *Set-Cover–Based Algorithms* balanced partitions can be created without the concern of creating one partition where most of the documents will be present. This makes these approaches more suitable than the Disjoint Sets algorithm when working with JSON documents. On the other hand, the SC Algorithms can provide bad partitioning if all the documents contain the same key-value pair. This can happen when there is a boolean key in the documents that in some documents is true and in others false. This would mean that the initial partitions will immediately be created with overlapping elements and from the initial phase replication of the documents will be introduced. With the execution of the second phase of the algorithms, the replication will only become worse. The effects of the replication are shown in Chapter 7.

In the Storm topology, both the PartitionCreator and the Merger bolt use the algorithms as they are represented in this section. PartitionCreator uses the first and the second phase for creating the local partitions and once the Merger receives the partitions from all PartitionCreators it uses the complete algorithm for creating the global partitions. As already mentioned, the Merger is respon-

sible for finding the most appropriate partition for the documents whose key-value pairs are not present in the already created partitions. For that reason, Algorithm 5 will be performed by the Merger when using the Set-Cover–based partitioning algorithm focused on optimizing the communication overhead.

Algorithm 5 Assigning Unseen Documents to Partitions

Input: Set PR of k partitions pr_i, Key-value pairs kv of new document
Output: Set PR of k final partitions pr_i
1: $pr_i = argmax_{\mathrm{pr}_j} |kv \cap pr_j|$ **and** $argmin_{\mathrm{pr}_j} \sum l_j$
2: $pr_i = pr_i \cup kv$
3: $PR.update(i, pr_i)$
4: **return** PR

Algorithm 5, as input receives the set of already created partitions PR and the set of key-value pairs kv of the document for which an appropriate partition needs to be found. The new document will be assigned to the partition pr_i that has the most elements in common with the set of key-value pairs kv of the document and at the same time the smallest load (Algorithm 5, Lines 1–2). By choosing the partition in this manner, the algorithm makes sure that the acceptable replication factor of the documents is preserved, while at the same time enabling an even distribution of the documents across the partitions.

5.3 Association Groups Based Algorithm

The next approach for creating partitions from the set of documents is based on finding *Association Groups* (AG). This approach differs from the already analyzed partitioning algorithms. Instead of finding the connected documents, this approach focuses on analyzing the elements within the documents. By evaluating the elements it draws conclusions about the patterns of occurrence of the elements within the documents and it uses these conclusions to group the documents that have the same combinations of elements. Once the documents are organized in association groups, the final partitions are created. When creating the partitions the algorithm focuses on providing partitions that are equally balanced in terms of load and partitions that have as lower replication as possible. Unlike the traditional association rule mining, we do not use the support and confidence to eliminate groups that occur by chance. We rely on the documents and if the elements form a connection in at least one document, they will be considered as a valid association. The reasoning that we make behind this choice is that we want our approach to provide results in real time without storing too much information and for that purpose, we make a trade-off between the actual importance of an association group and the speed of the algorithm.

The partitioning algorithm based on association groups is executed in two phases. In the first phase, the association groups are calculated from the key-value pairs of the JSON documents and in the second phase, the partitions are created based on the computed association groups. For finding the association

groups, we rely on two relationships between the key-value pairs of the documents, the equivalence, and the implication relationship. Equivalence Groups are formed by calculating the equivalence relationship in all the documents and once the equivalence groups are computed it is determined which equivalence groups can be connected by the implies relationship and in that way, the final association groups are formed. The equivalence and implication relationships are defined in the following way:

Let $D = \{d_1, .., d_n\}$ be the set of all documents and $kv_i = \{k_1 : v_1, .., k_m : v_m\}$ be the set of key-value pairs for the document d_i. We say that the key-value pairs $(k_1 : v_1, k_2 : v_2)$ form an equivalence group eg_1 if whenever $k_1 : v_1$ appears in a document d_i then also $k_2 : v_2$ appears and whenever $k_2 : v_2$ appears then also $k_1 : v_1$ appears. Let eg_1 and eg_2 be two computed equivalence groups. We say that eg_1 implies eg_2 if whenever the key-value pairs for eg_1 appear in a document then also the key-value pairs for eg_2 appear, but the key-value pairs of eg_2 can also appear in documents individually without the presence of the key-value pairs of eg_1.

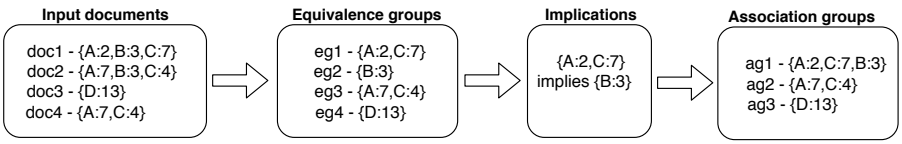


Figure 5.1: Finding Association Groups

Figure 5.1 represents an example for finding Association Groups. From the set of documents, firstly, the equivalence groups are created. Following the explanation for equivalence qroups, the algorithm determines that $eg1 = \{A : 2, C : 7\}$ is an equivalence group since $A : 2$ and $C : 7$ always appear together in all the documents. After the equivalence groups are calculated, they are used for identifying the implication relationship. It is determined that $eg1$ implies $eg2$ since in every document where $\{A : 2, C : 7\}$ appear also $B : 3$ appears. By grouping together the equivalence groups that satisfy the implies relationship, the final association groups $ag1, ag2$ and $ag3$ are produced.

The realization of the first phase responsible for finding the Association Groups is presented in Algorithm 6. The algorithm starts by creating a map referred to as $kvInD$, where the key will represent a set of documents and the value will be the key-value pairs that appear in that set of documents. The $kvInD$ map is created by iterating over the set of all key-value pairs and collecting the documents in which that key-value pair appears (Algorithm 6, Line 3). Once the documents for a key-value pair are found, the map $kvInD$ is updated (Algorithm 6, Line 3). With the creation of the map, actually the equivalence groups have been created since all the key-value pairs that appear in the same set of documents have been group together. For creating the association groups only the key-value pairs are required and not the documents where they appear. For that reason, the set EG is created where all the values from the $kvInD$ map are added (Algorithm 6, Lines 7–10). The association groups are created by iterating over the set of equivalence groups EG and checking which equivalence group is implied by another equivalence group from the set EG (Algorithm 6, Lines 12–22). If it is determined that equivalence group eg_i implies eg_j, then

Algorithm 6 Association Groups Based Algorithm - Phase 1: Finding Association Groups(**AG-P1**)

Input: Integer k, Set S_KV of kv-pairs sets s_kv_i, Set of kv-pairs KV, Set of documents D

Output: Set AG of association groups

1: $kvInD = new\ Map()$ //documents in which the key-value pairs appear
2: **while** $KV \neq \emptyset$ **do**
3: $docs_i = \cup_i D_i,\ kv_i \in D_i$
4: $kvInD.put(docs_i, (kvInD.get(docs_i) \cup kv_i))$
5: $KV = KV \setminus kv_i$
6: **end while**
7: $EG = \{\}$ //final equivalence groups
8: **for** $docs_i$ **in** $kvInD$ **do**
9: $EG = EG\ \cup\ kvInD.get(docs_i)$
10: **end for**
11: $AG = \{\}$ //final association groups
12: **for** eg_i **in** EG **do**
13: $ag_i = eg_i$
14: **for** eg_j **in** EG **do**
15: **if** eg_i **implies** eg_j **then**
16: $ag_i = ag_i\ \cup\ eg_j$
17: $EG = EG \setminus eg_j$
18: **end if**
19: **end for**
20: $l_i = |\cup_i D_i|, d_i \in ag_i$
21: $AG = AG\ \cup\ ag_i$
22: **end for**
23: **return** AG

eg_j is removed from EG since we do not want to create association groups with overlapping key-value pairs (Algorithm 6, Line 15–17). Additionally, for every association group AG the load is calculated by counting the number of documents in which the key-value pairs of AG appear (Algorithm 6, Line 20).

Algorithm 7 outlines the assignment of association groups to the required k partitions. For assigning the association groups to the partitions the same approach as in the Disjoint Sets algorithm (Algorithm 1) is used. The algorithm starts by first selecting the association group ag_i with the largest load (Algorithm 7, Line 3). If the initial k partitions are not yet populated, ag_i is assigned to the next free partition and the number of free partitions is reduced (Algorithm 7, Lines 5–8). If there are no more free partitions, ag_i is assigned to the partition that has the least load (Algorithm 7, Lines 9–12). The same steps are repeated until all the association groups have been assigned to partitions.

The partitioning algorithm based on association groups works with the single key-value pairs and tries to find pairs that appear together in all the documents or pairs that imply the appearance of other key-value pairs. This is different compared to the other partitioning approaches because they work with the whole set of key-value pairs for every document. This approach ensures that all the

Algorithm 7 Association Groups Based Algorithm - Phase 2: Finding Partitions(**AG-P2**)

Input: Integer k, Association Groups AG
Output: Set PR of k final partitions pr_i

```
1:  PR = {} // final partitions
2:  while AG ≠ ∅ do
3:      ag_i = argmax_{ag_j} l_j
4:      AG = AG \ ag_i
5:      if k > 0 then
6:          pr_k = ag_i
7:          PR = PR ∪ {pr_k}
8:          k = k − 1
9:      else
10:         pr_i = argmin_{pr_j} Σ_{ag_k ∈ pr_j} l_k
11:         pr_i = pr_i ∪ ag_i
12:     end if
13: end while
14: return PR
```

key-value pairs that appear together throughout all the documents will always reside on the same partition, thus, sending the documents to multiple machines is avoided. The association group with the largest load will always be assigned to the partition with the least load and with that good load balancing is provided. If the documents do not change in terms of their key-value pairs, meaning that the same combination of key-value pairs will continue to appear throughout the new documents that arrive, there will not be a need for updating the association groups and with that also the partitions will remain the same.

On the other hand, the set of key-value pairs for one document may be distributed in multiple partitions. This means that potentially every document will need to be sent to more than one machine. As the number of documents increases, there is a higher possibility that the same key-value pair appears in combination with different key-value pairs in different documents. This fact will make the already calculated association groups incorrect. Nevertheless, the problems mentioned above can be avoided by re-calculating the association groups once the quality of the partitions drops below a predefined threshold.

For using the association groups based partitioning algorithm in our proposed topology (Section 4.2), that is working in a distributed environment, a modification of the partitioning algorithm is made. Every instance of the PartitionCreator bolt will be responsible for finding the association groups from the allocated set of documents. This means that the PartitionCreators will execute only the first phase (Algorithm 6) of the partitioning algorithm. Once the PartitionCreators calculate the association groups for the given sliding window, they will emit the association groups to the Merger. When the Merger receives the association groups from every PartitionCreator, before continuing with the creation of the partitions, the Merger needs to consolidate the association groups and create the final set of association groups. Since the PartitionCreators are oblivious for the whole set of documents on which the partitioning is performed, the

job of the Merger is to create the correct association groups out of the received association groups.

Algorithm 8 Association Groups Based Algorithm - Consolidation of Association Groups in Merger Bolt (**AG-MC**)

Input: Association Groups AG
Output: Set of refined association groups AG

```
1:  sort(AG) //sorting of the association groups in descending order
2:  for ag_i in AG do
3:      for ag_j in AG do
4:          if ag_j ⊂ ag_i then
5:              AG = AG \ ag_j
6:          end if
7:      end for
8:  end for
9:  for ag_i in AG do
10:     ag_i = ag_i \ kv_i,  kv_i ∈ ag_j and ag_j ≠ ag_i
11: end for
12: return AG
```

The algorithm performed by the Merger for computing the final association groups is outlined in Algorithm 8. Initially, the algorithm starts by sorting the association groups in descending order based on the number of key-value pairs that they have (Algorithm 8, Line 8). In the next step, the algorithm removes all association groups that are a subset of another association group (Algorithm 8, Lines 1–7). For eliminating the duplicate key-value pairs, the algorithm iterates over the sorted association groups and if a key-value pair is contained in an association group with less key-value pairs, then the key-value pair is removed from the association group with more elements (Algorithm 8, Line 9–11). As a result, the Merger creates the final, consolidated association groups out of all the association groups received from the PartitionCreators. For creating the k partitions, the Merger uses the second phase (Algorithm 7) of the partitioning algorithm based on association groups and it provides as input the association groups computed with Algorithm 8. For the assignment of the documents with unseen key-value pairs to the already created partitions, the Merger uses the same algorithm as in the Set-Cover–based approach (Algorithm 5).

Chapter 6

Join Algorithms for Local Execution

When all the JSON documents have been efficiently partitioned and the potentially joinable JSON documents have been located on the same machine, the next step is to perform join over the potentially joinable documents, in order to determine the actually joinable documents. Two documents d_1 and d_2 are joinable if they have at least one common key-value pair such that none of their keys have more conflicting values than the specified threshold θ. For example, let $\theta = 1$, meaning that if two documents have at least one shared key for which they both have different values, they will not be provided as a result for joinable documents. The documents $d_1 : \{a : 2, b : 3\}$ and $d_2 : \{b : 3, c : 1\}$ are joinable documents based on the key-value pair $b : 3$, but the documents $d_1 : \{a : 2, b : 3\}$ and $d_3 : \{a : 2, b : 4\}$ are not joinable documents because the key b has conflicting values in the two different documents even though the documents share the key-value pair $a : 2$.

For computing the joinable documents based on our proposed definition for a join, we have investigated several joining algorithms. We have analyzed the suitability of the algorithms for our scenario and provided the respective advantages and disadvantages. In the following sections explanation of the investigated join algorithms together with our proposed implementation will be provided.

6.1 Nested Loop Join (NLJ)

The simplest, textbook algorithm for finding join partners is the *Nested Loop Join*. We will use this algorithm as a starting point for performing join over the JSON documents. Nested Loop Join is performed by iterating over all the documents in two nested loops and for every pair of documents it performs a check whether they can be joined. Through the check it is determined whether the documents satisfy our proposed definition for a join.

Algorithm 9 Nested Loop Join (**NLJ**)

Input: Set of JSON documents D

```
1: for dOutside in D do
2:     for dInside in D do
3:         if joinable(dOutside, dInside) then
4:             storePair(dOutside, dInside)
5:         end if
6:     end for
7: end for
```

Algorithm 9 shows how the JSON documents can be joined using a Nested Loop Join. The method *joinable*() checks if the two documents share at least one common key-value pair and they do not have any keys with conflicting values.

The Nested Loop Algorithm is an example of an algorithm that can be easily understood and implemented in a straightforward manner. The complexity of the algorithm is $O(|D|^2)$, where $|D|$ represents the number of JSON documents on which the algorithm is performed.

The disadvantage of this approach comes from the number of input documents. When the number of documents increases, the execution time of this algorithm will be unacceptably high. Since the main focus is to perform joins on data streams, the expected number of input documents would always be higher. For example, if the Nested Loop Join algorithm is performed on 50,000 documents, it means that 50,000 * 50,000 (2,500,000,000) iterations will be performed and if we assume that every check of two documents is performed in 500 ns the algorithm will need 1,250,000,000,000 ns or approximately 20 minutes to be executed. This execution time is not acceptable for a system that needs to join enormous amounts of documents, for that reason a new, more adequate join approach is required.

6.2 Hash-Based Join (HBJ)

One of the most effective join algorithms for performing equi joins in relational database management systems (RDBMS) is the hash-based join algorithm. The main idea is to build a hash table based on the distinct values of the join attribute where all the tuples that share a value will be part of the same bucket list in the hash table. Even though in RDBMS the joinable documents can be immediately obtained as values for a key in the hash table, for our setting this is not the case. A check must be performed, where all the documents that have conflicting values for the same key will be marked as NOT joinable. Thus, the hash table gives us only the potentially joinable documents, and to obtain the actually joinable documents one more check needs to performed over all the documents for a given key in the hash table. An outlook of the proposed hash-based join algorithm is given in Algorithm 10.

First, the hash table needs to be built. The hash table will have as key a key-value pair and as value all the documents that have that key-value pair (Algorithm 10, Lines 2–9). As a result, groups of potentially joinable documents will be created. Next, the actually joinable documents need to be determined (Algorithm 10, Lines 11–22). This is performed by iterating over the groups of potentially joinable documents and calling the method *joinable*() (Algorithm 10, Line 17). The method *joinable*() checks if the two documents have conflicting values for the same key. If the documents pass the join test, the resulting map *joinableDocs* of joinable documents will be updated (Algorithm 10, Line 18). The map *joinableDocs* has as key a document and as a value list of all the documents that are actually joinable with that document. Even though it is not shown in this algorithm, we can go one step further and reduce the number

Algorithm 10 Hash-Based Join (**HBJ**)

Input: Set of JSON documents D
Output: Map called $joinableDocs$ of pairs of joinable documents

```
1: hashTable = new Map()
2: for d in D do
3:     kvPairs = d.getKVPairs()
4:     for kvPair in kvPairs do
5:         existingKVPairs = hashTable.get(kvPair)
6:         existingKVPairs.add(d)
7:         hashTable.put(kvPair, existingKVPairs)
8:     end for
9: end for
10: joinableDocs = new Map()
11: for kvPair in hashTable.keys() do
12:     potentiallyJoinable = hashTable.get(kvPair)
13:     for i = 0; i < potentiallyJoinable.size(); i++ do
14:         dOutside = potentiallyJoinable.get(i)
15:         for j = i + 1; j < potentiallyJoinable.size(); j++ do
16:             dInside = potentiallyJoinable.get(j)
17:             if joinable(dOutside, dInside) then
18:                 storeJoinable(joinableDocs, dOutside, dInside)
19:             end if
20:         end for
21:     end for
22: end for
```

of groups of potentially joinable documents. This can be achieved by grouping the key-value pairs that appear in the same documents. This means, that the map storing the potentially joinable documents $hashTable$, as a key will have a set of key-value pairs.

The HBJ algorithm is easily comprehensible and its benefits are clear. By grouping the documents for a key-value pair, the number of documents that will be iterated is reduced. If the documents have fairly distinct key-value pairs, then this algorithm will provide satisfactory execution time.

On the other hand, if there is a common key-value pair that is present in most of the documents we again have the same problem as with the NLJ algorithm, too slow execution time. If the documents include a key with a boolean value that is true in 50 percent of the documents and false in the other, the same problem arises. If the cardinality of the document set is 50,000 we would have 25,000 documents that have a boolean value true and 25,000 documents with boolean value false. This means that for the list of documents in the hash table for the key-value pair boolean true 25,000*25,000 (625,000,000) iterations will be performed. If we assume the same execution time of 500 ns for performing a check for joinable documents, the algorithm will need approximately 5 minutes to find the joinable documents only for boolean true. The HBJ algorithm has a higher memory consumption in comparison to the NLJ since it additionally stores a hash table of the documents.

Analysis of the documents needs to be performed before choosing this approach. If the documents have fairly distinct key-value pairs then this algorithm can provide the needed results in satisfactory time, if this is not the case a more efficient, faster approach is required.

6.3 FPTree-Based Join

From the explanation of the previous approaches, we can conclude that a new algorithm for joining the JSON documents is needed. This algorithm needs to operate with a substantial amount of documents and provide the needed results in acceptable time. What we can take away from the previous join algorithms is that there is a need for more compact and concrete representation of the JSON documents that instead of storing the key-value pairs that appear in multiple documents several times, will just inform that they belong to different documents and store them once. Additionally, we can draw the lesson that there is a need for an approach that will avoid checking pairs of documents that are not joinable or will reduce this number of checks because they directly affect the execution time of the algorithm.

Section 2.4 explains the novel and effective way of storing frequent items in Frequent Pattern Trees, proposed by Han et al. [13]. The FPTrees store the information in a compact way. This is accomplished by defining a strict ordering of the items which is followed every time when a new item set is added to the tree. This means that if two item sets have the same items, they will be written only once in the tree and through a counter, it will be shown that those items belong to two different item sets. Additionally, through the helper links between the nodes that have the same item-name, the FPTree can be iterated in a much faster way in comparison to the standard breath-first and depth-first traversal techniques [13]. The FPTrees represent the basis for the implementation of a more suitable join algorithm.

For the FPTree used in our setting, first, a definition of the ordering of the key-value pairs in the JSON documents is specified. Second, a build phase is performed where the key-value pairs of the JSON documents are added as nodes in the FPTree and additionally, a helper table is built. The helper table provides linking between all the nodes that have the same label or in our setting the nodes that represent the same key-value pair. After the construction of the FPTree, a traversal algorithm is executed that provides as output the documents that can be joined with each other. In the following subsections, a detailed explanation of these phases is provided and the benefits of the FPTree-based joining algorithm are stated.

6.3.1 FPTree Design and Creation

Let $D = \{d_1,\ d_2,\ ..., d_n\}$ be the set of all JSON documents and $kv_i = \{kv_1,\ kv_2,\ ..., kv_m\}$ the set of key-value pairs that belong to document d_i. Given the set of all documents D a FPTree is built by adding the key-value pairs of the documents as branches in the tree.

As an example, the JSON documents that are shown in Table 6.1 are analyzed. First, a fixed ordering of the key-value pairs is required. The fixed ordering is used for ordering the key-value pairs of every document.

Document ID	Key-Value Pairs	Ordered KVPairs
d_1	$\{a:3,\ b:7,\ c:1\}$	$\{b:7,\ a:3,\ c:1\}$
d_2	$\{a:3,\ b:8\}$	$\{b:8,\ a:3\}$
d_3	$\{a:3,\ b:7\}$	$\{b:7,\ a:3\}$
d_4	$\{c:1\}$	$\{c:1\}$
d_5	$\{b:8,\ c:2\}$	$\{b:8,\ c:2\}$

Table 6.1: Set of JSON Documents

The method for determining the fixed ordering of the key-value pairs is presented in Algorithm 11. One pass over the JSON documents is required to determine the distinct keys that appear in the document set, to count the number of documents in which every key appears and to count the number of distinct values for every key (Algorithm 11, Lines 2–10). The information for the keys are stored in the list *keys*. The next step is to define the fixed ordering of the keys present in the list *keys*. The keys are ordered in descending order based on the number of documents in which every key appears.

Algorithm 11 Ordering of Key-Value Pairs

Input: Set of JSON documents D

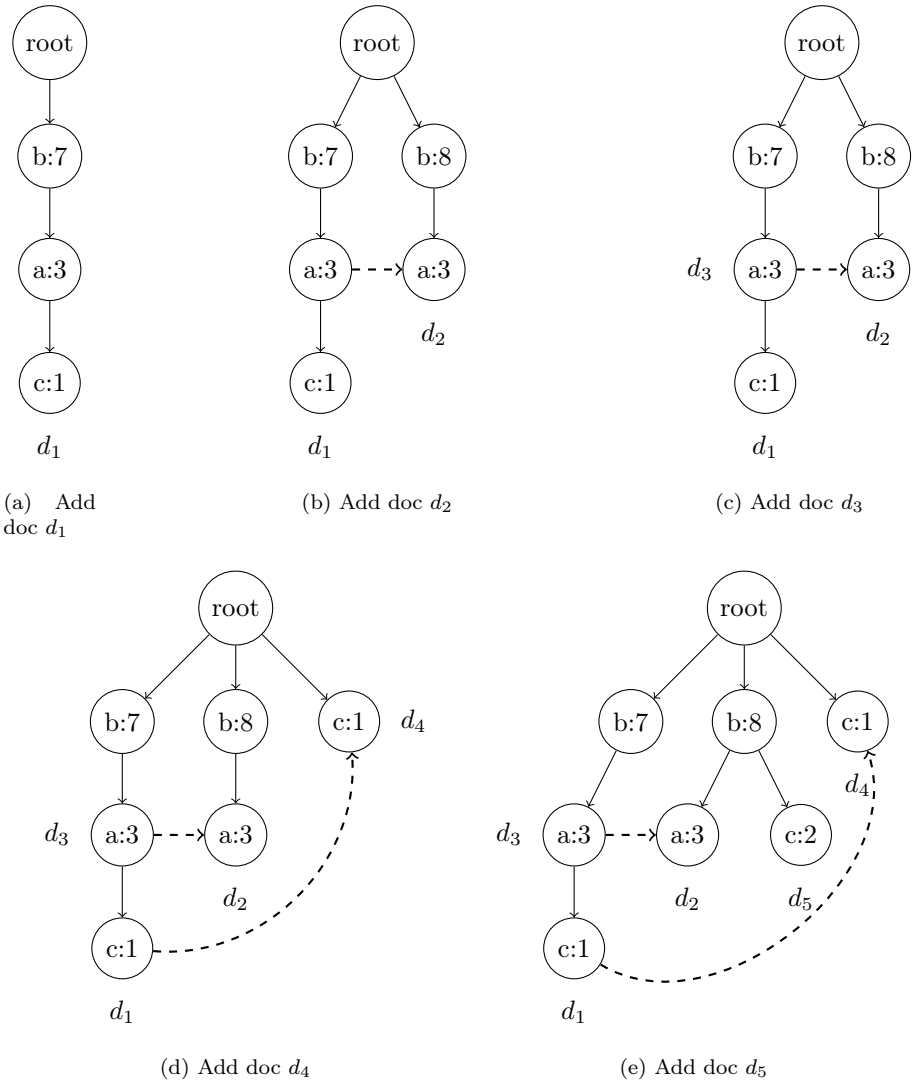
```
1:  keys = new List()
2:  for d in D do
3:      kvPairs = d.getKVPairs()
4:      for kvPair in kvPairs do
5:          keyObj = keys.getObjectForKey(kvPair.getKey())
6:          keyObj.incrementNumberOfDocs()
7:          keyObj.updateDistinctVals(kvPair.getValue())
8:          keys.update(keyObj)
9:      end for
10: end for
11: sorted = false
12: while !sorted do
13:     sorted = true
14:     for i = 0; i < keys.size() − 1; i++ do
15:         if keys[i].numOfDocs() < keys[i + 1].numOfDocs() then
16:             swap(keys[i], keys[i + 1])
17:             sorted = false
18:         else if keys[i].numOfDocs() == keys[i + 1].numOfDocs() then
19:             if keys[i].distinctVals() < keys[i + 1].distinctVals() then
20:                 swap(keys[i], keys[i + 1])
21:                 sorted = false
22:             end if
23:         end if
24:     end for
25: end while
```

If two keys appear in the same number of documents, the key that has a smaller number of distinct values will have a higher priority in the ordering. In Algorithm 11, from Line 11 until Line 25, the code for the ordering of the key is represented. The check for which key appears in a higher number of documents is performed on Line 15 and if two keys appear in equal number of documents, their distinct values are compared on Line 19. Once the keys are ordered, the key-value pairs for every document can be ordered based on the fixed ordering and with the help of the fixed ordering, the documents will be correctly added to the FPTree.

Returning to the example, first, the number of documents in which every key appears is counted and as result, we have that the key 'a' appears in 3 documents, 'b' appears in 4 documents and 'c' appears in 3 documents. In the fixed ordering, the key 'b' will be in the first position since it appears in the most number of documents. The keys 'a' and 'c' appear in an equal number of documents, so the number of distinct values that they have is checked. The key 'a' has 1 distinct value and the key 'c' has 2 distinct values. This means that the final fixed ordering will be $b \rightarrow a \rightarrow c$. In the column named 'Ordered KVPairs' of Table 6.1, the ordering of the key-value pairs for every document is shown.

Once the ordering of the key-value pairs is found, the construction of the FPTree can be started. The set of JSON documents D needs to be traversed one more time in order to construct the FPTree. When creating the branches of the FPTree, information about the documents to which the nodes of the branch belong will be stored. This is accomplished by storing a list of document names in the last node for every document. For every branch created, a unique *branch id* will be produced. With the branch id, during the execution of the join algorithm, branches that do not satisfy the join definition will be pruned. The second traversal will represent the final traversal of the full set of documents. The helper table consisting of the links between the nodes that represent the same key-value pair is built in parallel with the creation of the FPTree.

In the beginning, the FPTree is empty and the algorithm starts by adding a root node labeled '*root*'. The second traversal of the set of documents D leads to the next document d_1 that needs to be added to the tree. Since the '*root*' does not have any children, the set of key-value pairs of the document d_1 is added as a branch in the FPTree. In the last node of the branch for document d_1, information that the nodes come from document d_1 is stored by storing the name of the document in a list. When adding the key-value pairs of the document d_1, the key-value pairs are attached to the tree based on the defined fixed ordering. The next document for adding to the tree is d_2. By checking the children of the '*root*', it is concluded that there is no child labeled with $b : 8$, which is the first key-value pair of document d_2. For that reason, a new branch is created consisting of the key-value pairs of d_2. When adding the key-value pairs of the document d_2, the algorithm detects that the key-value pair $a : 3$ of document d_2 also appeared in document d_1. As a result, a link in the helper table is added between the nodes labeled with $a : 3$, representing the key-value pairs of documents d_1 and d_2. When the next document d_3 is selected for adding to the FPTree, the algorithm infers that the 'root' already has a node labeled equally as the key-value pair $b : 7$.



(a) Add doc d_1

(b) Add doc d_2

(c) Add doc d_3

(d) Add doc d_4

(e) Add doc d_5

Figure 6.1: Constructing an FPTree

For that reason, instead of creating a new branch, the algorithm traverses the branch that starts with the node labeled with $b : 7$. The next key-value pair from document d_3 that needs to be added to the tree is $a : 3$. The children of the node $b : 7$ are analyzed and it is determined that the node $b : 7$ already has a child node labeled with $a : 3$. That means that the node labeled with $a : 3$ is selected as the next starting point for the continuation of the algorithm. Since document d_3 has no more children, the list of documents in node $a : 3$ is updated by adding the name of document d_3. Because the 'root' has no child labeled with $c : 1$, the key-value pairs of document d_4 are added in a new branch of the tree. For document d_5, the 'root' already has a child labeled with $b : 8$ and this node is selected for the next steps of the algorithm. Because the node $b : 8$ has no child labeled with $c : 2$, a new node is created whose parent will be the node

$b : 8$. Additionally, the list of documents for node $c : 2$ is updated by adding the name of document d_5. Following the adding of the documents explained above, the outlook of the FPTree after every step is presented in Figure 6.1.

One can observe that the FPTree proposed by Han et al. [13] is modified so that it can satisfy our needs. The counter for the number of occurrences of the elements is eliminated because there is no need for storing the number of documents in which a particular key-value pair has appeared. Instead, a list of documents for every node is stored, indicating the exact documents in which that node and the appropriate parent nodes have appeared. With the creation of the FPTree, the problem of iterating over too many documents is solved. By reducing the stored number of key-value pairs, the search algorithm needs to cover fewer elements. Additionally, with the helper links between the equally labeled nodes, a possibility for faster traversal of the tree is provided. Based on these benefits, an algorithm that can provide join partners in a reasonable time is constructed. The details for the implementation of the join algorithm are given in the next section.

6.3.2 FPTreeJoin Algorithm for Finding Join Partners

For producing the join partners while efficiently traversing the constructed FP-Tree, we propose the *FPTreeJoin* algorithm. The suggested algorithm utilizes the benefits of the FPTree to navigate immediately to branches that hold potential candidates for join while at the same time ignoring the branches that have conflicting elements. The FPTreeJoin algorithm consists of two parts. In *Part 1*, based on the key-value pairs of the current document, we make use of the helper table to identify branches that hold potentially joinable documents. In *Part 2*, the nodes of the collected branches from *Part 1* are traversed and as a result, the documents that fulfill the join test are produced. Additionally, in

Algorithm 12 FPTreeJoin

Input: Set of JSON documents D, Map called $helperTable$
Output: Map called $joinableDocs$ of pairs of joinable documents

```
joinableDocs = new Map()
for d in D do
    kvPairs = d.getKVPairs()
    keys = d.getKeys()
    blacklist = new List() //blacklist for branches
    for kvPair in kvPairs do
        nodesForKvPair = helperTable.get(kvPair)
        for node in nodesForKvPair do
            if isCandidate(node, kvPairs, keys, blacklist) = true then
                joinableDocs.add(joinDocs(node, kvPairs, keys, blacklist))
            end if
        end for
    end for
end for
return joinableDocs
```

Part 2, all branches that have keys which are in conflict are added to a **blacklist**. The blacklist is used in *Part 1* to condense the set of branches with potentially joinable documents. The blacklist contributes directly to the performance of the algorithm, because branches that unquestionably store documents that cannot be joined will be immediately ignored.

The structure of the FPTreeJoin algorithm is presented in Algorithm 12. For every document d, the keys and the key-value pairs are gathered and additionally, a blacklist that will consist of the branches that should be ignored is created (Algorithm 12, Lines 3–5). Based on the nodes for every key-value pair, gathered through the *helperTable* (Algorithm 12, Line 7), it is firstly determined whether the branch for every node should be further examined for locating joinable documents by calling the method *isCandidate*() (Algorithm 12, Line 9). By calling the method *isCandidate*(), *Part 1* of the FPTreeJoin algorithm is performed. If the check for the branch is satisfied, the nodes of the branch are used for delivering the join partners of the input document d. This is accomplished by calling the method *joinDocs*() (Algorithm 12, Line 10). Through the method *joinDocs*(), *Part 2* of the FPTreeJoin algorithm is realized. The blacklist for the document d is provided as input to the methods *isCandidate*() and *joinDocs*(). The methods update the blacklist once they find a branch that violates the join definition stated in Section 1.1.

Algorithm 13 FPTreeJoin - Part 1: Checking Candidate Branches

Input: Node *node* from the FPTree, Set of key-value pairs $kvPairs$, Set of keys $keys$, List of branches to ignore *blacklist*
Output: Boolean value indicating whether the branch should be ignored

```
1:  function isCandidate
2:      if node.getParent() = 'root' then
3:          return true
4:      end if
5:      parentKvPair = node.parent.getKVPair()
6:      if keys.contains(parentKvPair.getKey()) and
        ¬kvPairs.contains(parentKvPair) then
7:          blacklist.addOrUpdateCounter(node.parent.branchId)
8:          if blacklist.getCounter(node.parent.branchId) ≥ θ then
9:              return false
10:         else
11:             return isCandidate(node.parent, kvPairs, keys, blacklist)
12:         end if
13:     else
14:         return isCandidate(node.parent, kvPairs, keys, blacklist)
15:     end if
16: end function
```

Algorithm 13 provides the implementation of the *isCandidate*() method responsible for realizing *Part 1* of the FPTreeJoin algorithm. This is a recursive method that checks whether the parents of the node provided as input are labeled with a key-value pair that might be in conflict with some of the key-value pairs of the input node (Algorithm 13, Line 6). If the parent node has a con-

flicting value for the same key, the *branchId* of the parent node is added to the blacklist and if the branch has more conflicting values than the specified threshold θ the method returns that the branch for the input node should not be considered for searching for join partners, otherwise the method continues by checking the next parent (Algorithm 13, Line 6–12). Branches can be eliminated in this way, because based on the construction of the FPTree we are certain that if a parent node has a conflicting value with the input document then all the children of that parent will also have conflicting values. On the other hand, this does not hold if we navigate through the FPTree from the top since every separate branch represents a different document and if one of the children of a node is in conflict it does not mean that all the other children will be in conflict too. If there are no conflicting elements, the method stops once the 'root' node is reached and informs that the branch can be further examined for locating join partners (Algorithm 13, Line 2–4).

Algorithm 14 FPTreeJoin - Part 2: Delivering Join Partners

Input: Node *node* from the FPTree, Set of key-value pairs *kvPairs*, Set of keys *keys*, List of branches to ignore *blacklist*
Output: Map *joinableDocs* of joinable partners

```
1:  function joinDocs
2:      if node.children.size() = 0 then
3:          joinableDocs.add(node.getDocumentNames())
4:          return joinableDocs
5:      end if
6:      for child in node.children do
7:          if keys.contains(child.getKey()) and
            ¬kvPairs.contains(child.getKVPair()) then
8:              blacklist.addOrUpdateCounter(child.branchId)
9:              if blacklist.getCounter(child.branchId) ≥ θ then
10:                 break
11:             else
12:                 joinableDocs.add(child.getDocumentNames())
13:                 joinableDocs.add(joinDocs(child, kvPairs, keys, blacklist))
14:             end if
15:         else
16:             joinableDocs.add(child.getDocumentNames())
17:             joinableDocs.add(joinDocs(child, kvPairs, keys, blacklist))
18:         end if
19:     end for
20:     return joinableDocs
21: end function
```

Part 2 of the FPTreeJoin algorithm is realized by the *joinDocs*() method outlined in Algorithm 14. The algorithm navigates through the branch provided as input in a top-down fashion and it determines whether the documents, represented by the nodes of the input branch, are join partners for the document provided as input. The algorithm iterates over the children of the branch (Algorithm 14, Line 6) and if the join test is satisfied (Algorithm 14, Line 7), the documents represented by the nodes are gathered and the algorithm continues

to navigate down through the branch (Algorithm 14, Lines 16–17). If the join test is unsatisfied, the *branchId* of the current child node is added to a blacklist (Algorithm 14, Line 8). If the join test has been violated more than θ times by the same branch, the consecutive children of the branch are ignored (Algorithm 14, Lines 9–10). The method will be repeated until there are no more children of the branch and as a result, all the documents that satisfy the join test will be returned (Algorithm 14, Lines 2–5).

6.3.3 Distributed Implementation

The compact way in which the documents are stored by the FPTree together with the execution time of the FPTreeJoin algorithm, which outperforms both the Nested Loop Join and the Hash-Based Join as shown by our experiments (Chapter 7), make this the only algorithm that we have used in the Storm topology.

Returning to the explanation of the topology in Section 4.2, it is stated that the Joiner is responsible for performing the actual join of the documents. This means that the Joiner will be responsible for creating the FPTree and realizing the FPTreeJoin algorithm. Since multiple instances of the Joiner can be created, a join for a larger number of documents can be executed in parallel. The creation of the FPTree is initiated by a time trigger which means that once the Joiner receives a tick tuple, the creation of the FPTree is commenced, realizing the approach explained in Section 6.3.1. When the FPTree is constructed for the current window of documents, the FPTreeJoin algorithm that executes the approach explained in Section 6.3.2 is started. The found join partners are emitted to a CSV file and the Joiners continue to store documents until the next time trigger is received.

Chapter 7

Experiments

The experiments are grouped in two categories, experiments that were performed in a distributed environment, where the goal was to test the chosen approaches for large volumes of documents, and local tests, were the focus was on determining which approaches should be implemented in the distributed setting by identifying the approaches that are most suitable for working with JSON documents. For the local tests, the partitioning algorithms explained in Chapter 5 and the join algorithms explained in Chapter 6 are compared. For the distributed environment, the impacts of the two implemented partitioning approaches on topology are analyzed. The data provided as input, in both the local and the distributed setting, was generated using the NoBench data generator [10].

7.1 Setup

The topology presented in this thesis was implemented using Java 8 and Apache Storm 1.2.2. For carrying out the experiments, our topology was deployed on a cluster of 12 servers, all running on Linux (Ubuntu 14.04.5) operating system. Every machine in the cluster has 2 processors with hexa-core Intel Xeon E5-2603 v4 CPU @ 1.7 GHz and 128GB RAM.

7.2 NoBench Data

The NoBench data generator [10] gives us the possibility to use an acknowledged benchmark for testing our topology. Through the NoBench generator, we were able to test the suitability of our approaches when working with documents that have different types of values. Initially, the NoBench data generator creates an array of JSON objects and in our work every JSON object represents a separate JSON document. The creation of the JSON array is initialized by calling the Python script that implements NoBench. When executing the script, one can specify the required number of documents as an input parameter of the script. For the different types of tests, different generation of the documents was used. For the local tests, the NoBench script was called once and the tests for every approach were performed repeatedly on the same corpus of documents. By storing the documents, it was possible to test the correctness of all the proposed approaches. For the experiments in the distributed environment, for every test, the Python script was executed and once the approaches were finished, the documents were lost. In this way, we were able to simulate a real data-stream with high velocity and volume.

7.3 Configuration Parameters

For testing the performance of our topology several settings for different parameters were examined. These parameters directly affect the components of the proposed topology.

- **Number of partitions** k – the number of partitions is directly connected to the number of Joiners, which means that by specifying the number of partitions also the number of Joiners is determined. By increasing the number of partitions, the load for every instance of the Joiner is minimized but on the other hand, the replication can be increased by spreading the joinable documents on different machines. For that reason, an appropriate value for the number of partitions needs to be chosen such that a balance between these two measures will be established. In our experiments, the tests were performed for $k = 6$, $k = 8$, $k = 10$ and $k = 20$ partitions.
- **Number of PartitionCreators** P – the number of PartitionCreators determines how many machines will be responsible for finding the initial partitions when implementing the Set-Cover–based algorithms or the number of initial association groups for the Association Groups based algorithm. In our test, the number of PartitionCreators was varied between $P = 4$, $P = 6$ and $P = 10$.
- **Join Threshold** θ – this value specifies the number of allowed conflicting values for the same key when checking if two documents are joinable. Even though it is possible to vary the join threshold θ, for the purpose of this thesis we always use $\theta = 1$ as the join threshold.
- **Sliding Window Size** w – the size of the sliding window specifies the number of documents that will be used in the PartitionCreators for creating the initial partitions. Through the size of the window, the communication overhead is directly affected. If the window is too small, it would mean that many documents will have attributes that are not present in the k partitions and as a result, the Assigners will send them to the Merger. For our experiments, the size of the sliding window was set to 40 and 60 percent of the total number of input documents.

In the following sections, the effects of these parameters in both the local and distributed environment are examined. Mainly, the experiments are focused on reporting the processing load, replication factor and execution time of the algorithms. For calculating the processing load and replication factor, the documents received by every instance of the Joiner bolt are analyzed. If not specified otherwise, we use default values for the parameters which are $k = 8,\ P = 6,\ \theta = 1$ and $w = 60\%$.

Additionally, the number of instances used for the Assigner bolt is 6 and for the JSONReader spout 2 instances are created. The Assigners emit every document that has an unseen key-value pair to the Merger. In the Merger, the assignment of the previously unseen documents to the partitions is commenced every 20 seconds, realized by sending a tick tuple to the Merger. In the Joiners, the joinable documents are computed and emitted to a CSV file every 2 minutes, which is realized by sending a tick tuple to the instances of the Joiner bolt.

7.4 Local Experiments

In the local measurements, the algorithms were implemented only using Java 8 and not in a Storm topology. There was a need to carry out these experiments because the partitioning approaches proposed by Alvanaki and Michel [5] were focused on analyzing Twitter tweets which compared to our JSON documents have fewer attributes. We wanted to determine whether the approaches are suitable before we realize them in Apache Storm. In this section, the Disjoint Sets, Set-Cover–based algorithms, and the Association Groups based algorithm explained in Chapter 5 are compared and results for the processing load, replication factor and execution time are presented. Additionally, the execution time of the join algorithms explained in details in Chapter 6 is compared and the consequent results are presented. For the tests realized in this section, the partitioning and joining algorithms are performed on the whole set of documents provided as input, meaning that the concept of sliding windows is not used in the local experiments.

7.4.1 Processing Load

In the local setting, the processing load is defined as the number of documents that were assigned to every partition through the used partitioning algorithm. To measure the inequality of the processing load between the created partitions, we use the *Gini coefficient*. The Gini coefficient is a widely used metric for computing statistical dispersion. A Gini coefficient with value 0 indicates that the elements are equally distributed among the partitions, whereas a Gini coefficient with value 1 indicates the worst possible distribution of the documents across the partitions.

Figure 7.1 presents the results for the load distribution in the local setting. The experiments for the load distribution were performed on a corpus of 50,000 documents. As it can be observed, all partitioning approaches except the Disjoint Sets provide acceptable load balancing of the documents among the partitions. This is an expected result since the Disjoint Sets algorithm is not able to populate the number of available partitions. The documents generated with the NoBench generator have a boolean value which is either true or false. As a result, the Disjoint Sets algorithm creates only 2 partitions for the set of documents where both partitions have around 50 percent of the whole document set or roughly 25,000 documents. The remaining $k-2$ partitions are empty, without any elements, which leads to the very poor load balancing. Additionally, as the number of partitions increases, the load distribution for the Association Groups based partitioning algorithm becomes worse. This is because of the way in which the partitions are created. When creating the association groups, there will always be two association groups consisting of the boolean value true and false. This means that constantly two partitions that have approximately 25,000 documents will be created. The remaining $k-2$ partitions will be created using the other association groups and they will always have less documents than the partitions with the boolean values. As the number of partitions increases, the number of documents in the partitions storing the boolean values will remain

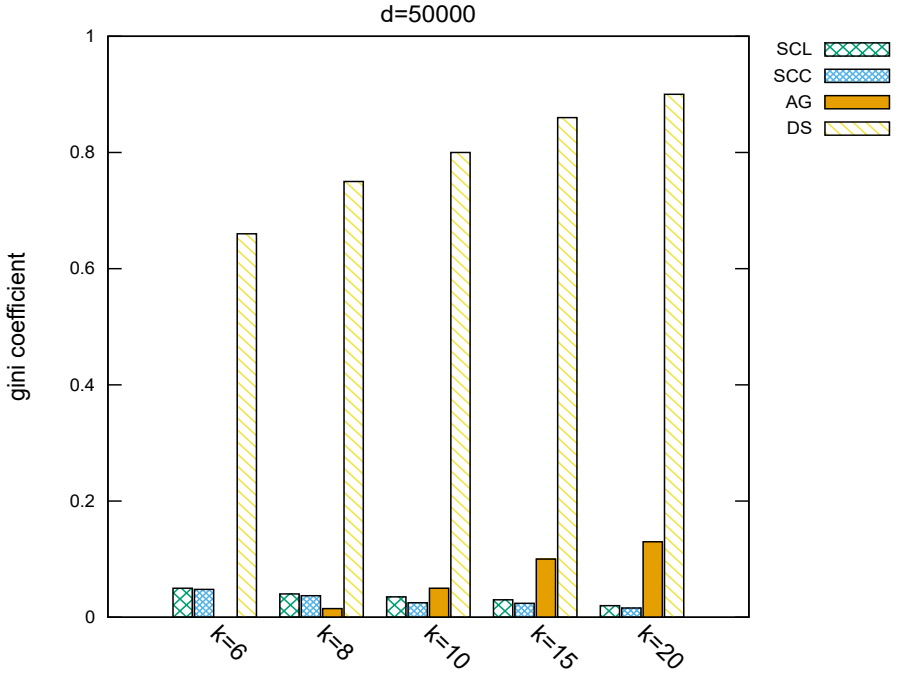


Figure 7.1: Processing Load - Local Measurements

the same, 25,000 documents, but the other partitions will have much less documents. Thus, the inequality between the partitions will be greater, contributing to the higher value for the Gini coefficient. On the other hand, in the Set-Cover–based algorithms, the improved load distributing is a result of replicating more documents across the partitions. This means that even though the partitions will be equally balanced in terms of load, they will need to process more documents and the consequent join algorithms will suffer. The exact measurements for the replication are presented in the next subsection.

7.4.2 Replication Factor

The replication factor in the local setting is computed based on the number of documents assigned to the partitions. Once the partitioning algorithm is performed and the partitions have been created, the number of documents on every partition is summed up and the sum represents the total number of replicated documents across the partitions. Through the calculation of the replication factor, we can infer the communication overhead of the partitioning algorithms if they were implemented in a distributed setting. This holds because a higher replication factor means that the same documents are present in more than one partition and as a result, in a distributed environment, they would be sent to more than one machine.

Figure 7.2 outlines how the change in the number of partitions affects the replication factor of the partitioning algorithms. One can immediately notice that

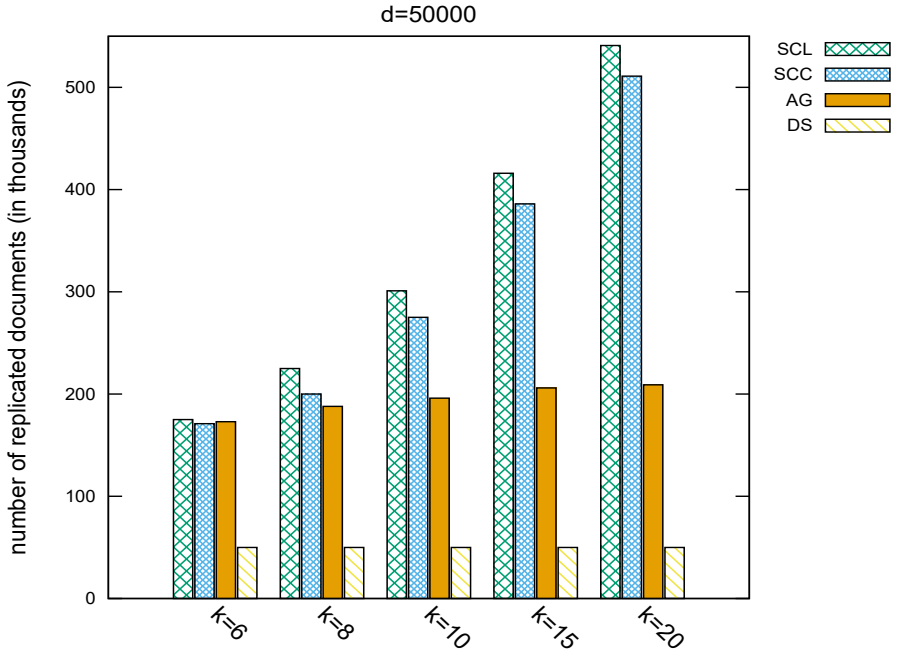


Figure 7.2: Replication Factor - Local Measurements

the Disjoint Sets algorithm has the most optimal replication factor which is the exact number of documents provided as input (50,000). This comes as no surprise since the Disjoint Sets algorithm groups the documents based on their shared key-value pairs and assigns every group to a different partition. Hence, there is no replication of the documents and every document belongs to one and only one partition. On the other hand, as already mentioned, only a small number of the total number of available partitions are populated with elements (in our tests 2 partitions) and all the remaining partitions are empty. This means that even though this approach provides the most optimal replication of documents, if it is implemented in a distributed environment, most of the available machines would stay idle which would result into losing all the benefits from the distributed computation. Additionally, based on the results we can conclude that as the number of partitions increases, the Association groups based partitioning algorithm has lower replication factor compared to the Set-Cover–based algorithms and this difference between the approaches becomes larger as the number of partitions increases. The Set-Cover–based algorithms create the partitions based on the whole set of key-value pairs for every document. This means that as the number of available partitions increases, it is more difficult for the Set-Cover–based algorithms to create partitions without overlapping key-value pairs. As a result, the documents will be replicated in multiple partitions which leads to the increased replication factor. On the other hand, the Association Groups based algorithm uses the key-value pairs from the documents and not the whole documents to create partitions. Therefore, the association groups that are computed for one set of documents will remain the same regardless of the changing number of partitions. Naturally, as the number of partitions

increases, the algorithm based on Association Groups will replicate some of the documents in multiple partitions but this number of replicated documents is much lower than the number of replicated documents with the Set-Cover–based algorithms.

7.4.3 Execution Time for Partitioning Approaches

The presented partitioning algorithms need to be implemented in a distributed environment and need to provide fast results to satisfy the requirements of real-time analysis. For that reason, we have analyzed the execution time of the respective algorithms.

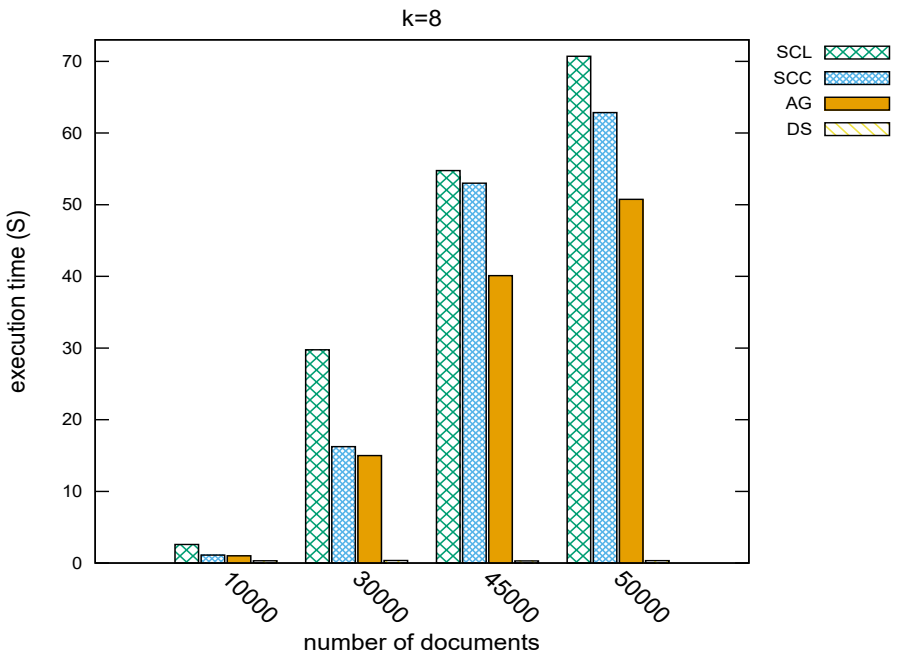


Figure 7.3: Partitioning Approaches Execution Time – Local Measurements

Figure 7.3 provides the results for the measurements of the execution time of the partitioning algorithms. Based on the results, the Disjoint Sets algorithm has the best execution time. This outstanding execution time is achieved based on the manner in which the algorithm determines the connected components. Finding one common key-value pair for two documents is enough for the algorithm to determine that they will be part of the same disjoint set, meaning that there is no need to check the additional key-value pairs of the documents. This way of creating the disjoint sets introduces an excellent stopping criteria that enables the algorithm to iterate over the documents in a fast manner and thus, provide the presented execution time. Additionally, based on the results, the execution time of the Association Groups based algorithm does not increase as drastically as the execution time of the Set-Cover–based algorithms when increasing the number of documents.

We would like to point out that the local execution time of the algorithms would also be acceptable for a distributed setting. If we assume that there is a cluster of 10 machines and every machine executes the partitioning algorithm for 50,000 documents, it would mean that most of the presented algorithms will deliver partitions for 500,000 documents in less than 60 seconds.

7.4.4 Execution Time for Local Join Approaches

For determining the most efficient join algorithm from the algorithms presented in Chapter 6, we have analyzed their execution time. The results from the measurements are outlined in Figure 7.4.

One can immediately observe that as the number of documents increases, the execution times of the Hash-Based Join (HBJ) and Nested Loop Join (NLJ) algorithms become unacceptably high for real-time analysis. For joining 50,000 documents, the NLJ algorithm requires more than 20 minutes whereas the HBJ algorithm needs close to 15 minutes. These times cannot be justified for any system that performs real-time analysis of data streams. On the other hand, for 50,000 documents, the FPTreeJoin algorithm finds the resulting joinable documents in less than 1.5 minutes. It is important to note that the time for the join algorithm based on FPTree presented in Figure 7.4 is the sum of the times for both creating the FPTree and performing the FPTreeJoin algorithm.

Based on these results, we have concluded that only the join algorithm based on FPTree will be implemented in our proposed Storm topology. Since every

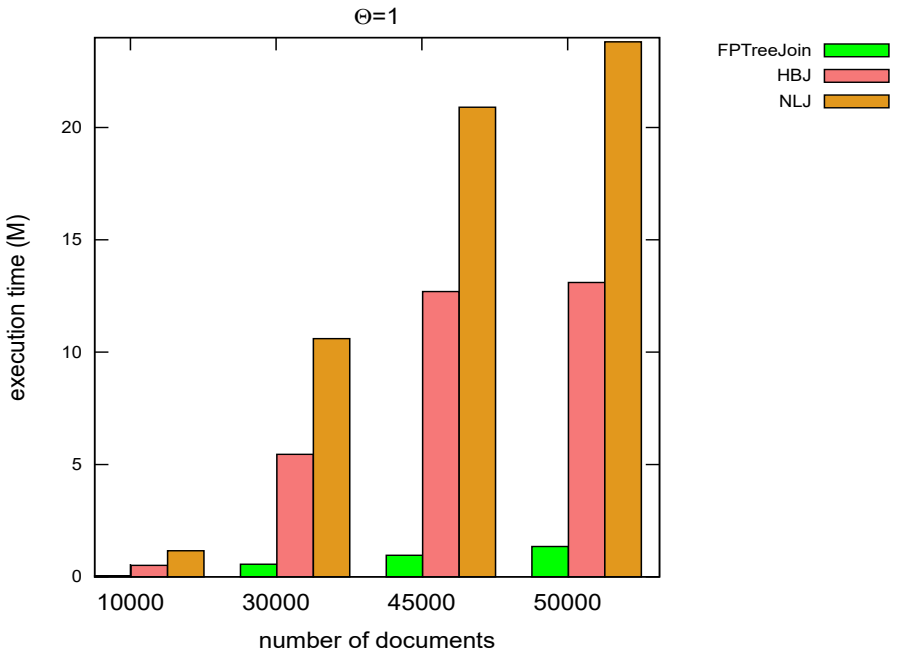


Figure 7.4: Join Approaches Execution Time - Local Measurements

Joiner bolt will execute the algorithms for both creating the FPTree and finding the joinable documents (FPTreeJoin), meaning that the join algorithm will not be distributed, the execution times for the join algorithm will be equal in the distributed environment as in the local setting. For that reason, the focus in the distributed environment will be on analyzing the implemented partitioning algorithms and not the implemented join algorithm.

7.5 Experiments in Distributed Setting

As a result of the experiments in the local setting, we have determined that only the association groups based partitioning algorithm and the set cover based partitioning algorithm focusing on communication overhead will be implemented in the Storm topology. Even though the disjoint sets algorithm provides good replication factor, it cannot be implemented for the JSON documents used because it requires an additional algorithm to split the enormous connected components. For finding the join partners, only the join algorithm based on FPTrees was implemented.

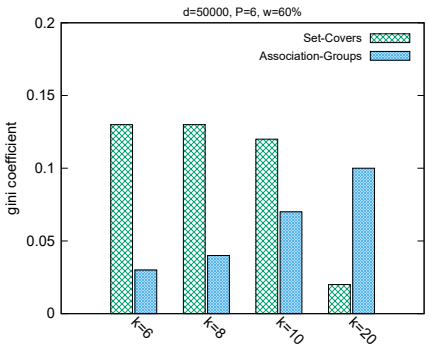
In the following subsections, results for the measurements of the processing load and replication factor will be presented.

7.5.1 Processing Load

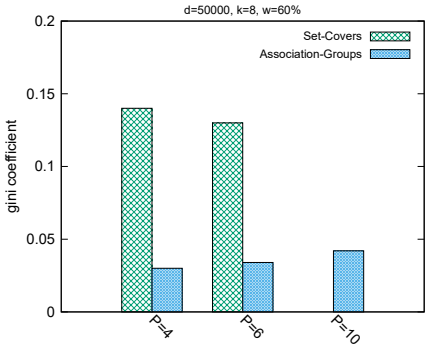
The processing load in the distributed setting was computed based on the number of documents that were received by every instance of the Joiner bolt. For measuring the inequality of the processing load, the Gini coefficient was used.

Figure 7.5 provides the information for the processing load when varying the different configuration parameters. Based on the results, it can be concluded that both approaches provide satisfiable load balancing, even though the Association Groups approach is slightly better. From Figure 7.5a, one can observe that as the number of partitions increases, the Association Groups approach has worse load balancing. The reasoning is the same as changing the number of partitions in the local setting explained in Subsection 7.4.1. Additionally, compared to the local setting, in the distributed environment the Association Groups approach has improved processing load for the same number of documents. This is a result of the consolidation algorithm (Algorithm 8) performed in the Merger bolt that groups together the local association groups computed by every PartitionCreator and provides an efficient partitioning of the documents. For the Set-Cover–based partitioning, by increasing the number of documents, the load distribution is improved and as it can be seen on Figure 7.5b, for 10 instances of the PartitionCreator this approach has the best Gini coefficient possible which means that all the machines operate on the same number of documents.

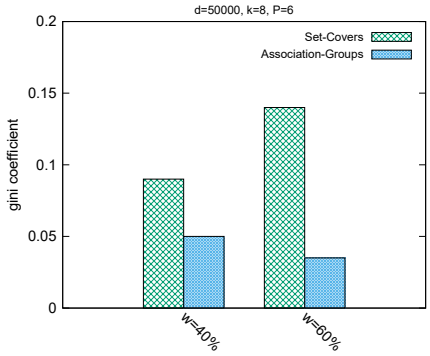
Based only on the processing load, one might assume that both approaches provide acceptable results since the difference between the Gini coefficients of the two different approaches is not that significant. To get the bigger picture, we additionally represent the maximal number of documents per machine. Figure 7.6 outlines this measurements. The Set-Covers approach, no matter the



(a) Varying Partitions



(b) Varying Partitioners



(c) Varying Window

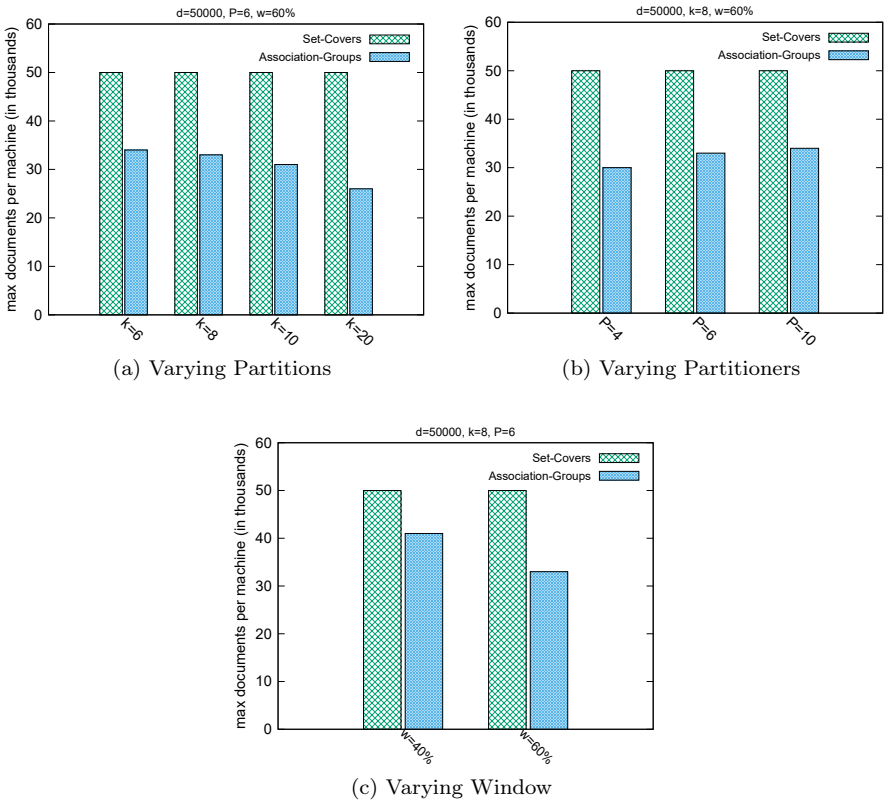
Figure 7.5: Processing Load

number of partitions or PartitionCreators used, constantly has a machine that operates on the whole set of documents provided as input (50,000). Returning to the results for the processing load, in order to obtain that low measurement for the Gini coefficient, all the machines need to process a similar number of documents. This means that in the Set-Cover–based algorithm, the number of documents processed by every machine is close to the number of documents provided as input. Furthermore, the best case scenario for the Set-Covers approach presented in Figure 7.5b, is not a result of having a good partitioning approach but actually is a result of sending all documents to every machine. The same can be accomplished without the use of a partitioning algorithm where every Assigner would send the documents to ever Joiner. This means that the configuration parameters have a great impact on the Set-Cover–based approach. If not chosen correctly, the algorithm will provide the worst possible partitioning of the documents which will be the same as not using any partitioning algorithm.

From Figure 7.5c, we can conclude that by increasing the sliding window, the Association Groups based approach achieves better load balancing. On the other hand, for the Set-Cover–based approach the opposite holds. When working with a smaller window over the documents, the Set-Covers approach has a difficulty to assign the new documents to the existing partitions. This results into creating

overlapping partitions by replicating the documents. The better Gini coefficient is due to the large number of replicated documents.

Through this measurements we show that the Association Groups based partitioning approach is more suitable for vast JSON documents. The Set-Cover–based algorithm, independent of the setting for the configuration parameters, always creates partitions, such that there is at least one machine working on the full set of input documents. Additionally, we show that the better load balancing for the Association Groups based approach is not a result of sending all documents to every machine but is a result of the efficient computation of the partitions.



(a) Varying Partitions

(b) Varying Partitioners

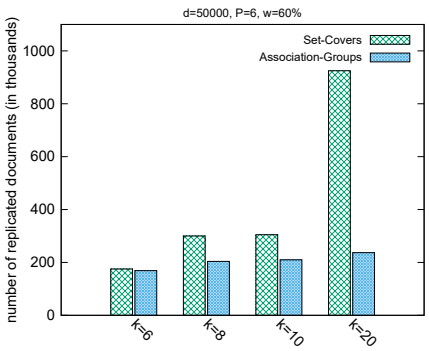
(c) Varying Window

Figure 7.6: Maximal Number of Documents per Machine

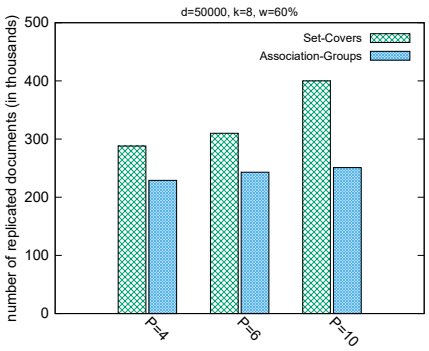
7.5.2 Replication Factor

The replication factor was computed based on the number of documents sent to every instance of the Joiner bolt. The changes in the replication factor with respect to the different settings of the various configuration parameters are presented in Figure 7.7. The configuration parameter that affects the replication factor the most is the number of partitions. As expected, by increasing the number of partitions both partitioning approaches have increased replication of

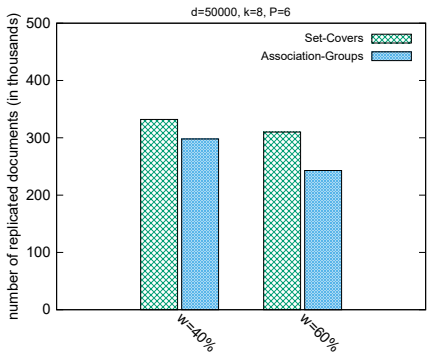
documents. The Association Groups approach handles the increase in the number of partitions acceptably by creating partitions without many overlapping key-value pairs. On the other hand, the replication factor of the Set-Covers approach suffers dramatically. This is a result of the way in which the partitions are created when using the Set-Covers algorithm. By having more available partitions, it is more difficult for the Set-Covers algorithm to assign key-value pairs to them without creating overlapping partitions. Additionally, from the results in Figure 7.7b, for 10 instances of the PartitionCreator, the Set-Covers algorithm has the worst possible replication as a result of sending every document to every machine. By varying the window of documents used, the replication factor changes. It is not a surprise, because if the partitioning algorithm is created on more documents more appropriate partitions are created and the problem of assigning many unseen documents based on their key-value pairs to new partitions is eliminated.



(a) Varying Partitions



(b) Varying Partitioners



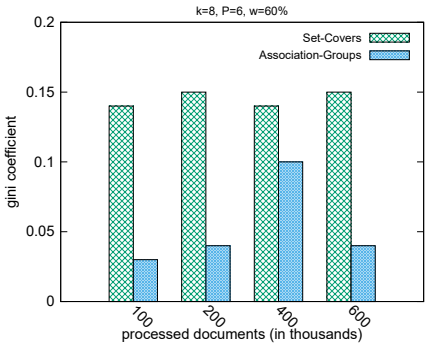
(c) Varying Window

Figure 7.7: Replication Factor

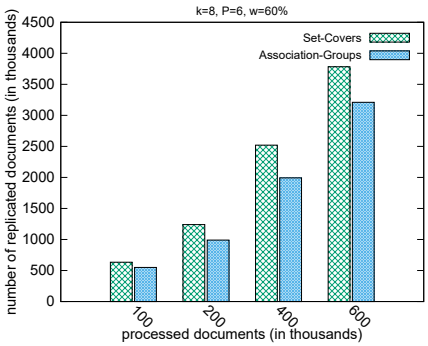
For a distributed system the replication factor is of great importance. Low replication factor means that fewer messages will be sent between the instances of the system, resulting in better network traffic and with that better execution of the whole system. Through the measurements for the replication factor, once again the suitability of the Association Groups based partitioning algorithm for partitioning JSON documents is reaffirmed.

7.5.3 Handling High Volume of Documents

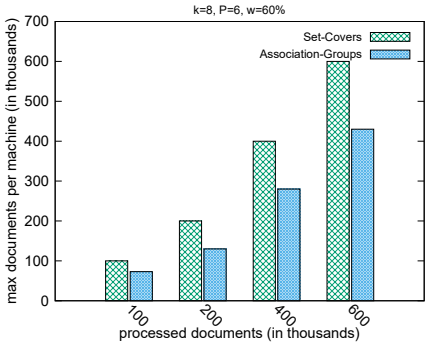
In Figure 7.8, we provide measurements about the load distribution and replication factor, when changing the number of documents on which the proposed system operates. Through these measurements, we asses the suitability of the Association Groups based partitioning algorithm when working with large data streams and we verify that our approach continues to outperform the Set-Cover–based partitioning algorithm.



(a) Processing Load



(b) Replication Factor

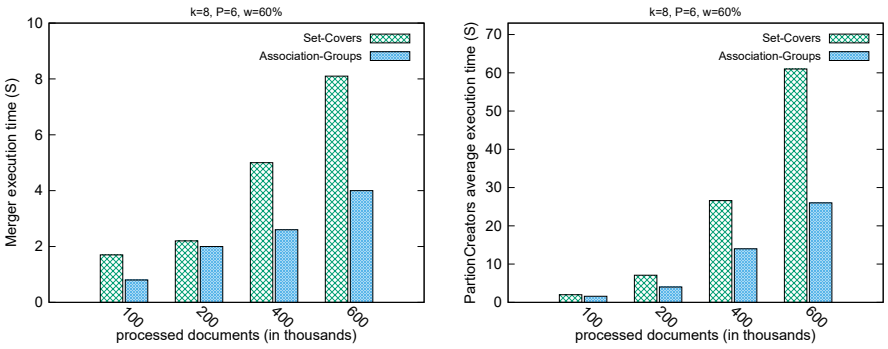


(c) Maximal Number of Documents

Figure 7.8: Changing the Number of Documents

In Figure 7.8a, the measurements for the processing load are outlined. As it can be observed, the Association Groups based partitioning algorithm, constantly, for the increasing numbers of documents, provides better load distribution than the Set-Cover–based approach. Additionally, by analyzing the results from Figure 7.8c, it is shown that the better load balancing is not a result of sending the whole set of input documents to every machine but is a result of the superior and more efficient partitioning approach. The Set-Cover–based approach continues to suffer from the same problem of having the full set of documents on at least one machine. Furthermore, the Association Groups based approach retains better replication factor. On the other hand, the Set-Covers approach is constantly close to the worst possible replication of sending every document to every machine.

For providing the complete comparison of the partitioning approach, in Figure 7.9, we present the execution times of the two algorithms. Because the algorithms were split such that part of the approach was realized by the PartitionCreators and the other part by the Merger, we measure and report both the execution time of the algorithm implemented by the Merger and the PartitionCreators. As it can be observed, the Association Groups based approach performs faster than the Set-Covers approach and for 600,000 documents it is executed in less than 40 seconds. The main difference is in the algorithms used by the PartitonCreators, where for an increasing number of documents the dissimilarity between the execution times of the approaches becomes more significant.



(a) Merger Execution Time (b) PartitionCreators Execution Time

Figure 7.9: Changing the Number of Documents - Execution Time

7.5.4 Impact of Sliding Window

The sliding window determines the number of documents on which the partitioning will be performed. If the sliding window is too small and the new documents received by the topology have previously unseen key-value pairs, the quality of the partitions will be affected. This is due to the fact that the Merger will need to find the most suitable partition for the newly arrived documents, based on the used partitioning algorithm. As a result, overlapping partitions will be created which affect both the processing load and the replication factor, as shown by the previous experiments. It is important to note that the problem of changing partitions will occur when the documents received are constantly changing over time. Even though for some types of documents this might not be the case, the JSON documents generated by NoBench have exactly this characteristic.

In Figure 7.10, the number of newly arrived documents that are sent to the Merger for different sliding windows used is presented. As outlined, when using a 40% window (20,000 documents) for partitioning, almost all of the subsequent documents will be sent to the Merger. This means that the Merger will be receiving tuples constantly and at the same time performing the algorithm for assigning the tuples to the most appropriate partition. As a result, the Merger will represent a bottleneck, affecting the execution of the whole topology. On

the other hand, when we are using a 60% window over the documents (30,000 documents) for partitioning, the number of unseen documents sent to the Merger is drastically reduced. As a result, the Merger will be able to assign the unseen documents to the existing partitions and inform the Assigners about the updated partitions in a more acceptable time.

Having constantly changing documents, introduces a real need for an algorithm that will be able to check the current quality of the partitions, and based on a predefined quality threshold, determine whether new partitioning of the documents should be performed. At this point of time, the re-creation of the partitions has not been implemented in our topology, but in Section 8.1, we propose an algorithm that can accomplish this.

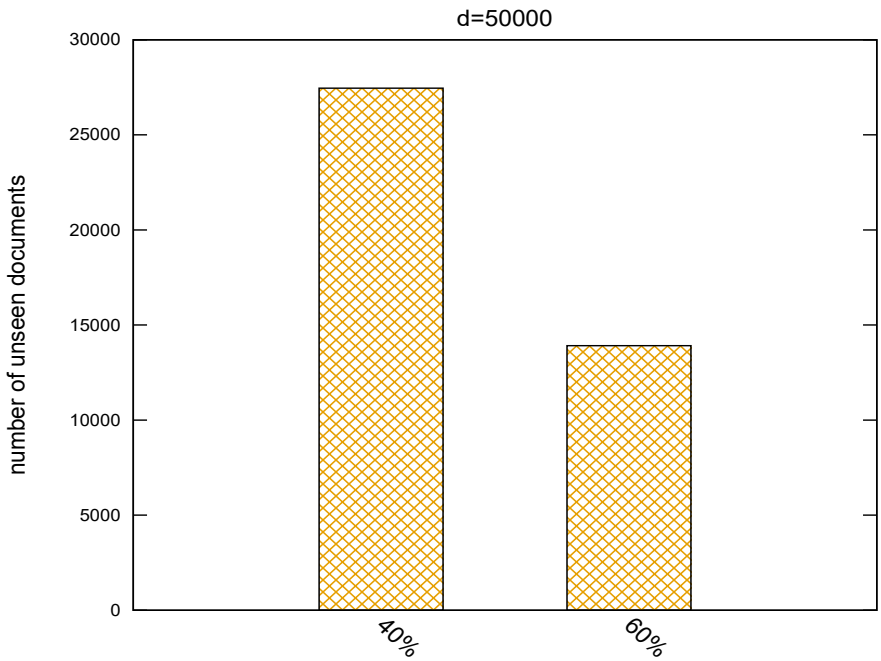


Figure 7.10: Unseen Documents for Varying Sliding Window

Chapter 8

Future Work

In this section, initially, we introduce ideas for improving our proposed topology that can contribute to improved load balancing and replication factor. These improvements can be realized through small changes, without much effort. Additionally, we discuss a possible extension of the topology, through which documents with similar key-value pairs will be provided as results.

8.1 Recalculating Partitions

As our main focus was to develop an approach for joining JSON documents, our topology currently does not consider that the documents might be changing over time. As a result, if the content of the newly arrived documents does not reflect the already created partitions, the quality of the partitions will be affected resulting in higher replication factor and unacceptable distribution of the documents across the machines. To avoid these problems, we need an approach that will monitor the quality of the partitions and once the quality falls below a predefined threshold, initiate the recalculation of the partitions.

For realizing the re-partitioning, we propose that the same approach presented by Alvanaki and Michel [5] should be followed since the same parameters for examining the quality of the partitions can be used in our approach. For implementing these improvements, only minor changes need to be made in the topology. The architecture will remain the same, meaning that the Partition-Creators will still be responsible for computing the initial partitions and the Merger will be responsible for creating the global partitions. The re-creation of the partitions should be initiated by the Assigners. The main reason for this is that the Assigner is the component that connects the partitions with the Joiners. For initiating the re-partitioning, the Assigners need to store some characteristics about the partitions. This information about the partitions should be provided by the Merger since that is the component that creates the final k partitions. The information provided about the partitions should be the maximal load $maxLoad$, representing the maximal number of documents assigned to one partition, and the average number of sent documents $avgDocs$, representing the number of documents that will be sent by the Assigners to the Joiners divided by the current window of documents used for partitioning. As a result, the Merger will emit not only the created partitions to the Assigners but also the maximal load and the average number of sent documents. These measurements will represent a threshold based on which the Assigners will determine whether a recalculation of the partitions should be initiated.

To detect whether the quality of the partitions is below a threshold, the Assigners additionally, need to store the number of messages (documents) that they have sent to the different instances of the Joiner bolt. Once a predefined

number of documents $coveredDocs$ has been analyzed by the Assigners, they will compute the maximal load $maxLoad'$ as the maximal number of messages sent to one Joiner, and the average number of sent documents $avgDocs'$ as the total number of messages sent to all Joiners divided by the covered documents $coveredDocs$. If $maxLoad'$ and $avgDocs'$ exceed $maxLoad$ and $avgDocs$ respectively, more than a predefined threshold, recalculation of the partitions will be initialized by sending a message to the PartitionCreators that they need to compute new partitions from the current window of documents. To enable the re-computation of the partitions, the PartitionCreators should constantly store a sliding window over the current documents.

8.2 Similarity Joins

Even though the focus of this thesis is to find joinable documents based on the equi-join definition, because of the manner in which the topology is realized, we believe that there is a possibility to broaden the scope. By modifying the algorithms in the already existing components, we can repurpose the topology, such that the focus will be on performing **similarity joins**. The similarity join has been widely analyzed when working with data streams. Mainly, the problem is defined as finding similarity only between strings using a different similarity function or performing a set similarity join, where the goal is to find the similarity between two sets based on their elements. Since our goal is to analyze JSON documents, only using the string similarity or the set similarity will not be enough. Instead, we would like to continue with the same strategy, where initially, partitions will be created and for every partition either one type of similarity or maybe a combination of several similarity join approaches will be used. We believe that by using multiple approaches, more suitable results for analyzing sizable JSON documents will be provided.

For the type of similarity that we propose, the keys and the values of the JSON documents should be considered separately. We propose that the similarity should be defined on the values in the JSON documents. This means, that we would say that two documents are similar if, for the same keys, the documents have values that satisfy the similarity threshold. For distributing the documents efficiently among the machines, a partitioning algorithm should be used. The realized partitioning approaches can still be useful when performing similarity join. The main difference would be in the elements on which the partitioning will be performed. Instead of using the key-value pairs of the documents, the partitioning algorithms should use only the set of keys of the documents. Once the documents are partitioned based on their keys, a similarity join algorithm should determine if two documents are joinable by checking the similarity between the values for the same key. If the documents have more keys with similar values than a predefined threshold, they should be provided as resulting joinable documents.

In Table 8.1 an example set of JSON documents is shown. Through the use of a partitioning algorithm on the keys, the partitions $\{car, engine, color, doors, hp\}$ and $\{name, cuisine, location, phone, website\}$ should be created. As a result, the documents d_1 and d_2 will be assigned to one machine and the documents

Document ID	Key-Value Pairs
d_1	$\{car : Focus,\ engine : 1.6\ D,\ color : black,\ doors : 3\}$
d_2	$\{car : Ford\ Focus,\ engine : 1.6,\ hp : 123\}$
d_3	$\{name : MyDiner,\ cuisine : it,\ location : KL,\ phone : 0123\}$
d_4	$\{name : MyDiner,\ cuisine : italian,\ website : mydiner.com\}$

Table 8.1: Example Set of JSON Documents

d_3 and d_4 on another. The next step is to compute the similarity of the values between the shared keys of the documents. For documents d_1 and d_2, through the use of a similarity function it will be determined that for the key *car* the values $Ford$ and $Ford\ Focus$ are similar and for the key *engine* the values 1.6 D and 1.6 are similar. As a result, the approach will output the documents d_1 and d_2 as resulting joinable documents. The computation of the similarity between the values can be realized by using a similarity function such as the Jaccard Similarity, Edit Distance or any other suitable similarity function.

As outlined, the proposed idea for similarity join would have a similar architecture as the already realized topology. Again, there will be two main parts, partitioning of the documents and computing the joinable documents. The main difference will be in the realization of the components. Instead of using the set of key-value pairs, the partitioning algorithms will be performed on the set of keys of the documents. For performing the similarity join between the values, a new algorithm needs to be implemented in the Joiner bolt. At this point, we cannot speculate about the efficiency of the implemented join algorithm for performing similarity join since an additional comparison of the documents needs to be performed. The additional comparison can impact the execution time of the algorithm, making it inefficient for real-time analysis. Only by implementing the similarity join and performing appropriate experiments we can determine whether the existing approach is suitable or a new, innovative approach is required.

Chapter 9

Conclusion

The goal of this thesis was to provide an approach that will be able to compute join partners from a large corpus of JSON documents provided as input. To enable real-time computation of the joinable documents, the approach should be implemented in a distributed setting. For that reason, the documents should be efficiently distributed among the available machines such that the groups of documents allocated to different machines will have as few overlapping documents as possible and all the machines will handle an approximately equal number of documents.

As a result, the presented Storm topology was created. For partitioning the documents, a novel algorithm based on association groups was created. The idea behind the partitioning approach and the realization of the approach were explained in details. Through several experiments, it was shown that the proposed approach is more suitable for analyzing versatile JSON documents that incorporate a large number of key-value pairs than the already existing partitioning algorithms that focus on analyzing Twitter tweets. Additionally, the impact of the several configuration parameters on the topology was examined and concluded that by varying the parameters the effect on the topology are not as dramatic as with the other partitioning algorithms.

In our work, for the actual joining of the documents, an innovative approach based on Frequent Pattern Trees was introduced. The need for this algorithm was a result of the different join definition used. Since it is not possible to specify the join attribute in advance, the JSON documents need to be analyzed and based on their key-value pairs, the algorithm should determine if they can be joined. Additionally, we introduce another requirement for two documents to be joinable, which is that the documents cannot have more than θ conflicting key-value pairs. As a result of the join definition used, the conventional join algorithms proved to be inefficient. By directly comparing the standard join algorithms with our proposed join algorithm, we conclude that the Frequent Pattern Tree based algorithm provides the needed results in acceptable time whereas the other join algorithms are executed in a time that cannot be justified for real-time analysis.

Possible improvements of our topology were introduced, which could improve the quality of the partitions when working with JSON documents whose content constantly changes over time. Through the re-calculation of the partitions, better load balancing and replication factor can be achieved. Additionally, the possibility of extending our topology to handle similarity joins was examined and suggestions for future realization were provided.

Bibliography

[1] Apache Storm. `http://storm.apache.org/`. [Online; accessed 17-February-2019].

[2] Apache Storm Component Guide. `https://docs.hortonworks.com/HDPDocuments/HDP2/HDP-2.6.5/bk_storm-component-guide/content/storm-windowing-concepts.html`. [Online; accessed 18-February-2019].

[3] JavaScript Object Notation. `https://www.json.org/`. [Online; accessed 20-February-2019].

[4] F. N. Afrati and J. D. Ullman. Optimizing joins in a map-reduce environment. In *Proceedings of the 13th International Conference on Extending Database Technology*, pages 99–110, New York, NY, USA, 2010. ACM.

[5] F. Alvanaki and S. Michel. Tracking set correlations at large scale. In *SIGMOD Conference*, 2014.

[6] R. Ananthanarayanan, V. Basker, S. Das, A. Gupta, H. Jiang, T. Qiu, A. Reznichenko, D. Ryabkov, M. Singh, and S. Venkataraman. Photon: fault-tolerant and scalable joining of continuous data streams. In *SIGMOD Converence*, 2013.

[7] S. Blanas, J. M. Patel, V. Ercegovac, J. Rao, E. J. Shekita, and Y. Tian. A comparison of join algorithms for log processing in mapreduce. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data*, pages 975–986, New York, NY, USA, 2010. ACM.

[8] G. Bonnard Schonhorst, V. Paes, P. Balestrassi, A. Paiva, and P. Campos. Data mining association rules applied to supermarket transactional data modeling: a case study in brazil. 07 2017.

[9] P. Chardaire, M. Barake, and G. P. McKeown. A probe-based heuristic for graph partitioning. *IEEE Transactions on Computers*, 56(12):1707–1720, Dec 2007.

[10] C. Chasseur, Y. Li, and J. M. Patel. Enabling json document stores in relational systems.

[11] G. Cugola and A. Margara. Processing flows of information: From data stream to complex event processing. *ACM Comput. Surv.*, 44(3):15:1–15:62, 2012.

[12] M. Elseidy, A. Elguindy, A. Vitorovic, and C. Koch. Scalable and adaptive online joins. *In Proc. VLDB Endow.*, 7(6):441–452, Feb. 2014.

[13] J. Han, J. Pei, and Y. Yin. Mining frequent patterns without candidate generation. *In SIGMOD Rec.*, 29(2):1–12, 2000.

[14] B. Hendrickson and R. Leland. A multi-level algorithm for partitioning graphs. In *SC Conference(SC)*, volume 00, page 28, 12 1995.

[15] J. Kang, J. F. Naughton, and S. D. Viglas. Evaluating window joins over unbounded streams. In *Proceedings 19th International Conference on Data Engineering (Cat. No.03CH37405)*, pages 341–352, March 2003.

[16] B. W. Kernighan and S. Lin. An efficient heuristic procedure for partitioning graphs. *The Bell System Technical Journal*, 49(2):291–307, Feb 1970.

[17] S. Khuller, A. Moss, and J. S. Naor. The budgeted maximum coverage problem. *Information Processing Letters*, 70(1):39 – 45, 1999.

[18] Q. Lin, B. C. Ooi, Z. Wang, and C. Yu. Scalable distributed stream join processing. In *SIGMOD*, pages 811–825, New York, NY, USA, 2015. ACM.

[19] P. Mandave and S. Patil. Data mining using association rule based on apriori algorithm and improved approach with illustration. 01 2013.

[20] D. L. Quoc, I. E. Akkus, P. Bhatotia, S. Blanas, R. Chen, C. Fetzer, and T. Strufe. Approxjoin: Approximate distributed joins.

[21] P. Sanders and C. Schulz. Engineering multilevel graph partitioning algorithms. In *Algorithms – ESA 2011*, pages 469–480, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.

[22] P.-N. Tan, M. Steinbach, A. Karpatne, and V. Kumar. *Introduction to Data Mining (2Nd Edition)*. Pearson, 2nd edition, 2018.

[23] Y. Wei, R. Yang, and P. Liu. An improved apriori algorithm for association rules of mining. In *IEEE International Symposium on IT in Medicine Education*, volume 1, pages 942–946, Aug 2009.

[24] M. Zaharia, T. Das, H. Li, T. Hunter, S. Shenker, and I. Stoica. Discretized streams: Fault-tolerant streaming computation at scale. In *SOSP*, pages 423–438, New York, NY, USA, 2013. ACM.

Eidesstattliche Erklärung

Ich erkläre hiermit an Eides statt, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe.

Statement in Lieu of an Oath

I hereby confirm that I have written this thesis on my own and that I have not used any other media or materials than the ones referred to in this thesis.

Kaiserslautern, _____ _____

Date Signature